

Bachelorarbeit

Oleksii Baida
Matrikelnummer 7210384

Entwicklung eines Mikrocontrollersystems für Steuerungs- und
Überwachungsaufgaben innerhalb von Häusern

Bericht

2. März 2025

Inhaltsverzeichnis

1	Einleitung	3
2	Ziel des Projekts	4
2.1	Komponenten des Systems	5
2.2	Architektur und Datenfluss	5
2.2.1	Struktur der MQTT-Kommunikation	6
3	Grundlagen & Theorie	8
3.1	Hardware	8
3.1.1	Arduino Uno	8
3.1.2	ESP8266	8
3.1.3	ESP32	9
3.1.4	BME680	10
3.1.5	VCNL4040	10
3.1.6	Raspberry Pi	11
3.2	Kommunikationsprotokolle	11
3.2.1	HTTP	11
3.2.2	MQTT	11
3.2.3	UART	11
3.2.4	I2C	11
3.3	Software	11
3.3.1	PlatformIO	11
3.3.2	Python	11
3.3.3	FastAPI	11
3.3.4	SQLAlchemy	11
3.3.5	WebSocket	11
3.3.6	Web	11
4	Einrichtung der Hardwarekomponenten	12
4.1	Arduino	12
4.2	Raspberry Pi	12
4.3	Einrichtung der ESP-Module	12
4.3.1	ESP8266	16
4.3.2	M5Stick	17
4.3.3	ESP32	17
5	Softwareentwicklung	18
5.1	Datenbank	18
5.2	Webbasierte Schnittstelle (API)	22
5.2.1	API-Endpunkte	24
5.2.2	Sicherheit	24
5.2.3	Aufbau	25
5.3	MQTT-Client	27
5.4	WebSocket	28
5.5	Frontend	28
5.5.1	Dynamische Datendarstellung mit JavaScript	29
5.5.2	Styling	35

6	Ergebnisse und Diskussion	36
7	Quellen	37
	Abbildungsverzeichnis	38
	Programmcode	38

1 Einleitung

In einer Welt, die zunehmend von vernetzten Geräten und dem Internet der Dinge geprägt ist, wird die Entwicklung effizienter und benutzerfreundlicher Systeme zur Steuerung und Überwachung von Gebäuden immer relevanter. Im Jahr 2024 wurde in Deutschland die Anzahl von über 19 Millionen Haushalten, die ein oder mehrere smarte Geräte besaßen, verzeichnet. Es wird prognostiziert, dass sich diese Zahl innerhalb der nächsten drei Jahre verdoppeln wird [4].

Moderne Steuerungs- und Sicherheitssysteme tragen zur Effizienzsteigerung und Ressourcenschonung bei. Laut Günther Ohland, Vorstandsmitglied des Branchenverbands SSmarthome Initiative Deutschland“, ermöglichen diese Systeme eine Reduktion des Heizenergieverbrauchs um 20 bis 30 Prozent [5]. Die Kosten für die smarte Technik rechnen sich in der Regel nach zwei Jahren. Die Systeme übernehmen ein Teil der täglichen Aufgaben, wie das Ein- und Ausschalten des Lichts, die Regelung der Raumtemperatur oder das Aufräumen des Hauses etc. Der Aufgabenbereich der Systeme ist dabei nur nach den Bedürfnissen der Benutzerinnen und Benutzer abgegrenzt.

Im Rahmen meiner Bachelorarbeit wird ein System zur Steuerung und Überwachung des Hauses entwickelt. Das Ziel dieser Arbeit ist die Erstellung einer Schnittstelle, die die Interaktion des Benutzers mit den Geräten in seinem Haushalt ermöglicht und den Benutzer über gefährliche Vorgänge in seinem Haus informiert.

Im Rahmen der Entwicklung dieses Systems wurden die aktuellen Technologien zur Erstellung eines Webinterfaces und zur Kommunikation zwischen den Geräten eingesetzt. **TODO Kurz erklären was in Kapitel 2,3,4,5 ... erklärt wird**

2 Ziel des Projekts

Das im Rahmen dieser Arbeit entwickelte System stellt einen Prototyp für ein Smart-Home-System dar. Ziel des Projekts ist die Realisierung eines intelligenten Heimsystems, das verschiedene Technologien integriert und flexibel erweiterbar ist. Die Architektur des Systems ist modular aufgebaut, sodass die Geräte hinzugefügt und eingebunden werden können. Dies erlaubt eine skalierbare und anpassbare Nutzung in unterschiedlichen Anwendungsfällen.

Das entwickelte System dient der Überwachung und Steuerung von vernetzten Geräten innerhalb eines Hauses. Ziel des Systems ist es, Sensordaten zu erfassen, diese zu verarbeiten und eine benutzerfreundliche Interaktionsmöglichkeit zu bieten. Dabei kommen verschiedene Hardware- und Softwarekomponenten zum Einsatz, die eine nahtlose Kommunikation zwischen den einzelnen Modulen gewährleisten.

Das System erfüllt mehrere wesentliche Aufgaben:

1. **Sensordatenerfassung:** Die an den Mikrocontrollern angeschlossenen Sensoren übermitteln die erfassten Daten an dem zentralen Server.
2. **Sicherheitsaspekt:** Das System ist mit den Sensoren für die Erkennung des Brandes ausgestattet. Außerdem bietet das System gesicherter Zugang zu dem Haus.
3. **Datenverarbeitung und Speicherung:** Der zentrale Server stellt die empfangenen Daten zur Analyse bereit und speichert die Benutzer- und Geräteinformationen in der Datenbank.
4. **Benutzerinteraktion:** Über die Weboberfläche können die Nutzer das System konfigurieren, Messwerte einsehen und Aktoren steuern. Außerdem werden die Nutzer über das Brand im Haus informiert.
5. **Kommunikation:** Die Kommunikation zwischen den Komponenten erfolgt über das MQTT-Protokoll, wodurch eine effiziente und skalierbare Datenübertragung ermöglicht wird.
6. **Automatisierung:** Basierend auf den erfassten Sensordaten können bestimmte Vorgänge automatisch ausgelöst werden.

2.1 Komponenten des Systems

Das System besteht aus mehreren Komponenten, die zusammenarbeiten, um eine effiziente Steuerung und Datenerfassung zu gewährleisten:

- **Raspberry Pi:** Der zentrale Server. Stellt das lokale WLAN-Netzwerk bereit, hostet den MQTT-Broker und führt die Webanwendung aus.
- **Arduino:** Ausgestattet mit mehreren Sensoren, die Gefahren wie Feuer und Gas erkennen und den Zugang zum Haus sichern. Entsprechende Meldungen werden an den Server gesendet.
- **M5Stick:** Angeschlossen an die Temperatur- und Lichtsensoren und sendet die aufgezeichneten Daten auf den Server.
- **ESP32:** Angeschlossen an Display und Temperatursensor dient zur Anzeige der Umgebungsdaten und sendet diese an den Server.
- **Webserver:** Eine auf FastAPI basierende RESTful API, die Benutzern den Zugriff auf das System und die Steuerung von Geräten ermöglicht.
- **Datenbank:** Eine lokale Datenbank zur Speicherung von Benutzerinformationen und Gerätekonfigurationen.
- **Web-Anwendung:** Eine browserbasierte Benutzeroberfläche, die den Benutzern eine intuitive Steuerung und Visualisierung der Daten ermöglicht.

2.2 Architektur und Datenfluss

Das entwickelte System basiert auf einer modularen Architektur, die mehrere Komponenten integriert.

Der Raspberry Pi dient als zentraler Server des Systems. Der Minicomputer stellt ein WLAN-Netzwerk zur Verfügung und hostet den Webserver mit der Datenbank sowie den MQTT-Broker. Alle Benutzerinteraktionen, Sensordaten, Datenflüsse und Datenverarbeitungen finden auf dem Server statt. Das System ist somit stark zentralisiert und arbeitet nur lokal. Das bedeutet, dass sich alle Benutzer in einem lokalen Netzwerk mit dem Raspberry Pi befinden müssen.

Die Geräte verbinden sich mit dem WLAN und mit dem MQTT-Broker, die vom Raspberry Pi zur Verfügung gestellt werden. Jedes Gerät verfügt über eine eindeutige Geräte-ID. Die Sensoren senden ihre Daten an den MQTT-Broker, der auf dem Raspberry Pi läuft. Die Akteure abonnieren die Command-Topics mit der entsprechenden "Geräte-ID".

Im Kern des Systems befindet sich ein Webserver, der auf dem Raspberry Pi ausgeführt wird. Dieser Webserver stellt eine RESTful-API zur Verfügung. Für den Zugriff zu der API wurde eine Webseite aufgebaut. Die API bietet folgende Funktionen an:

- Authentifizierung und Autorisierung der Benutzer
- Verwaltung der Gerätekonfigurationen und Benutzerprofile
- Bereitstellung von Endpunkten zur Abfrage und Steuerung von IoT-Geräten
- Bereitstellung der Daten von den Geräten in Echtzeit
- Bidirektionale Kommunikation mit den IoT-Geräten

Die Benutzerauthentifizierung erfolgt über JSON Web Tokens (JWT). Bei der Anmeldung wird ein JWT erzeugt, das eine verschlüsselte Benutzer-ID enthält. Dieses Token wird auf der Seite des Benutzers gespeichert und bei jeder Anfrage an den Server gesendet, um die Identität des Benutzers zu verifizieren. JWTs haben eine festgelegte Lebensdauer, nach deren Ablauf eine erneute Anmeldung oder eine Token-Erneuerung erforderlich ist. Die Verwendung von JWT ermöglicht eine sichere, zustandslose Authentifizierung, da der Server keine Sitzungsdaten speichern muss.

Die Benutzerdaten und Gerätekonfigurationen werden in einer lokalen Datenbank gespeichert, die mit `SQLite` implementiert wird. `SQLite` hat den Vorteil, dass die gesamte Datenbank in einer einzigen Datei gespeichert wird, wodurch der Ressourcenverbrauch des Servers minimiert wird. Der Zugriff auf die Datenbank erfolgt über `SQLAlchemy`, eine leistungsfähige ORM-Bibliothek für Python. `SQLAlchemy` ermöglicht asynchrone Datenbankzugriffe, was die Leistung und Skalierbarkeit der Anwendung verbessert.

Die Geräte senden ihre Daten an den zentralen MQTT-Broker. Um die eingehenden Nachrichten effizient verarbeiten zu können, wird ein MQTT-Client mit Hilfe der Bibliothek `aiomqtt` implementiert. Diese Bibliothek verwendet Python `asyncio`, was eine nicht-blockierende, asynchrone Verarbeitung ermöglicht. Dadurch können Nachrichten empfangen und verarbeitet werden, ohne den Hauptprozess zu blockieren. Das verbessert die Reaktionsfähigkeit des Systems und ermöglicht die parallele Verarbeitung mehrerer MQTT-Nachrichten, was besonders wichtig für die Systeme mit mehreren Geräten ist.

Die Gerätedaten müssen den Benutzern in Echtzeit zur Verfügung gestellt werden. Zu diesem Zweck wird `WebSocket` verwendet. Es ermöglicht eine kontinuierliche und bidirektionale Datenübertragung zwischen Server und Client. Dadurch wird sichergestellt, dass die Daten sofort und ohne wiederholte Anfragen an den Benutzer übermittelt werden.

Der Benutzer benötigt eine Schnittstelle, um auf das System zugreifen zu können. Dazu wird eine Webseite in Form einer SPA¹ entwickelt. Eine SPA lädt einmal die grundlegende HTML-, CSS- und JavaScript-Struktur und aktualisiert dann dynamisch nur die relevanten Inhalte, ohne die gesamte Seite neu zu laden. Der Hauptvorteil einer SPA liegt in der verbesserten Nutzererfahrung, da Seitenwechsel nahezu verzögerungsfrei erfolgen. Außerdem wird die Serverlast reduziert, da nur die benötigten Daten über eine API geladen werden, anstatt komplette HTML-Dokumente zu übertragen. Dies führt zu einer schnelleren und reaktionsfähigeren Anwendung, was insbesondere bei Anwendungen mit Echtzeitanforderungen von Vorteil ist.

Die für die SPA entwickelte API kann auch von anderen Anwendungen, z. B. mobilen Anwendungen, genutzt werden. Dadurch können die Anwendungen auf einer gemeinsamen Backend-Struktur aufbauen, was die Wartung vereinfacht und die Wiederverwendbarkeit des Codes erhöht.

Die vorgestellte Architektur bietet eine effiziente, skalierbare und reaktionsfähige Struktur. Der Zugriff auf das System erfolgt über die API. Durch die Verwendung asynchroner Methoden werden Blockierungen vermieden. `WebSocket` stellt die empfangenen Daten ohne serverseitige Speicherung sofort zur Verfügung. Die SPA bietet eine schnelle und benutzerfreundliche Schnittstelle mit direktem Zugriff auf die API-Endpunkte. Diese Architektur garantiert eine hohe Performance und vereinfacht sowohl die Wartung als auch zukünftige Erweiterungen des Systems.

2.2.1 Struktur der MQTT-Kommunikation

Die im System verwendeten Geräte kommunizieren über das MQTT-Protokoll. Der MQTT-Broker wird auf dem Raspberry Pi ausgeführt.

¹Engl. Single Page Application

Die Geräte verbinden sich mit dem MQTT-Broker und senden ihre Nachrichten in individuelle Topics. Es gibt 4 Haupttopics, die für jedes Gerät mit der entsprechenden Geräte-ID erweitert werden. Die Tabelle 1 listet die Topics auf. ID wird von jedem Gerät ermittelt und beim Senden der Nachricht entsprechend gesetzt.

Topic	Nachricht	Beschreibung
handshake/ID	<code>"data_fields": ["temperature", "humidity"], "commands": ["light_turn_on", "light_turn_off"]</code>	In diesem Topic senden die Geräte ihre Konfigurationen.
data/ID	<code>"temperature": 23, "humidity": 75</code>	Hier werden die von den Sensoren erfassten Daten übertragen.
command/ID	<code>"light_turn_on"</code>	Dieses Topic wird von Geräten abonniert. Der in der Nachricht übermittelte Befehl wird vom Gerät ausgeführt.
alarm/ID	<code>"fire": "fire_start"</code>	Dies ist ein spezielles Topic für das Alarmsystem. Wenn der Alarm ausgelöst wird, wird die entsprechende Meldung in diesem Topic gesendet.

Tabelle 1: MQTT-Nachrichten

Beim Hinzufügen eines neuen Geräts zum System sendet der dem Broker eine Handshake-Nachricht in das Topic „**command/ID**“ mit dem Text „**handshake**“ gesendet. Das Gerät antwortet darauf im Topic „**handshake/ID**“ mit seiner Konfiguration in JSON-Format. Im Feld „**data_fields**“ werden die Felder angegeben, die von den Sensoren erfasst werden. Im Feld „**commands**“ werden die Befehle angegeben, die das Gerät ausführen kann. Diese Daten werden in der Datenbank gespeichert, können jedoch durch erneutes Senden der Handshake-Nachricht aktualisiert werden. Nach der Handshake-Nachricht werden in das Topic „**command/ID**“ ausschließlich Nachrichten mit gespeicherten Befehlen gesendet.

Die Nachrichten, die in das Topic „**data/ID**“ gesendet werden, müssen ein JSON-Format haben. Das erleichtert die Verarbeitung der Daten.

TOODOO Diagramm zum Zeigen der Kommunikation.

3 Grundlagen & Theorie

In diesem Abschnitt werden die technischen Spezifikationen der verwendeten Komponenten und Technologien detailliert beschrieben.

3.1 Hardware

Das Projekt umfasst verschiedene Hardware-Komponenten, darunter Mikrocontroller, Sensoren und einen Einplatinencomputer.

3.1.1 Arduino Uno

Der Arduino Uno ist ein Mikrocontroller-Board, das sich besonders für die Erfassung und Verarbeitung von Sensordaten eignet. Aufgrund seiner einfachen Programmierbarkeit und vielseitigen Schnittstellen wird es häufig in eingebetteten Systemen sowie in IoT-Anwendungen eingesetzt.

Das Herz des Boards bildet der Mikrocontroller ATmega328P, der über Flash-Speicher und EEPROMElectrically Erasable Programmable Read-Only Memory verfügt. Das EEPROM ermöglicht die nichtflüchtige Speicherung von Daten, so dass diese auch nach dem Ausschalten des Gerätes erhalten bleiben.

Die technischen Spezifikationen des Arduino Uno sind in Tabelle 2 dargestellt.

Modell	Arduino Uno
Mikrocontroller	ATmega328P
Taktfrequenz	16 MHz
Eingangsspannung	12 V
Digital Pins	14
Analoge Eingänge	6
Flash-Speicher	32 KB
SRAM	2 KB
EEPROM	1 KB
Kommunikationsschnittstellen	UART, SPI, I2C
USB-Schnittstelle	USB-B

Tabelle 2: Technische Daten des Arduino Uno

Im Rahmen dieses Projekts wird der Arduino Uno zur Erfassung und Verarbeitung von Sensordaten für die Branderkennung eingesetzt. Darüber hinaus übernimmt er die Steuerung des kontrollierten Zugangs zum Gebäude. Durch die Anbindung an verschiedene Sensoren ermöglicht er eine kontinuierliche Überwachung der Umgebung und kann im Gefahrenfall entsprechende Meldungen auslösen.

3.1.2 ESP8266

Der ESP8266 ist ein kompakter Mikrocontroller mit integriertem WLAN-Modul. Aufgrund seiner kompakten Größe, seines geringen Stromverbrauchs und seiner integrierten WLAN-Schnittstelle eignet er sich besonders für drahtlose Sensornetze und eingebettete Systeme.

Ein zentrales Funktionsmerkmal des ESP8266 ist seine integrierte WLAN-Funktionalität. Das Modul kann direkt mit einem WLAN-Netzwerk verbunden werden oder als Access Point ein WLAN-Netzwerk zur Verfügung stellen. Zusätzlich kann auf dem Mikrocontroller ein einfacher Webserver eingerichtet werden. Damit kann es sowohl als eigenständiger Mikrocontroller als auch als Wi-Fi-Erweiterung für andere Mikrocontroller, z.B. einen Arduino, verwendet werden.

Das Modul basiert auf dem ESP8266 Chip von Espressif Systems. Es verfügt über Flash, SRAM und EEPROM Speicher. Außerdem unterstützt es das UART-Kommunikationsprotokoll. Die technischen Spezifikationen des ESP8266-01 sind in der Tabelle 3 dargestellt.

Modell	ESP8266-01
Chip	ESP8266EX
Architektur	32-Bit
Taktfrequenz	80 MHz
Eingangsspannung	5 V
Flash-Speicher	1 MB
SRAM	80 KB
Kommunikationsschnittstellen	UART
WLAN-Standard	IEEE 802.11
Sicherheitsmechanismen	WEP, WPA, WPA2

Tabelle 3: Technische Daten des ESP8266

In meinem Projekt wird der ESP8266 als Schnittstelle zwischen dem Arduino und dem zentralen Server verwendet. Er ermöglicht die drahtlose Übertragung von Sensordaten an das Backend und kann auch Steuerbefehle empfangen, um entsprechende Aktionen auszulösen. Durch die direkte WLAN-Anbindung trägt der ESP8266 wesentlich zur flexiblen und ortsunabhängigen Systemsteuerung bei.

3.1.3 ESP32

Das ESP32 ist ein leistungsstarker Mikrocontroller von Espressif Systems. In der Literatur wird das Modul auch als SoC² bezeichnet, was bedeutet, dass alle Funktionen des Moduls auf einem Chip integriert sind.

Das ESP32 unterstützt WLAN und Bluetooth. Außerdem verfügt es über Schnittstellen wie I2C, UART, GPIOs, die eine flexible Integration mit Sensoren und anderen Peripheriegeräten ermöglichen.

Die technischen Spezifikationen des ESP32 sind in der Tabelle 4 dargestellt.

Modell	ESP32-WROOM-32
Architektur	32-Bit Dual-Core
Taktfrequenz	Bis zu 240 MHz
Betriebsspannung	3,3 V
Digitale GPIO-Pins	34
Analoge Eingänge	18
Flash-Speicher	4 MB
SRAM	520 KB
Kommunikationsschnittstellen	UART, SPI, I2C, CAN, I2S
USB-Schnittstelle	Mikro-USB
WLAN-Standard	IEEE 802.11 b/g/n
Bluetooth	BLE 4.2, Classic
Energiesparmodi	Deep-Sleep, Light-Sleep

Tabelle 4: Technische Daten des ESP32

²Eng. System-on-a-Chip, Deutsch: Ein-Chip-System

M5StickC

Der M5StickC ist eine kompakte Entwicklungsplattform, die auf dem ESP32 basiert. Er verfügt über ein integriertes Display, eine Batterie und zusätzliche integrierte Sensoren.

Der M5StickC verfügt über ein 1,14-Zoll großes LCD-Display, das die visuelle Ausgabe von Sensordaten oder Statusmeldungen ermöglicht. Außerdem enthält er eine integrierte IMU (Inertial Measurement Unit) zur Bewegungs- und Lageerkennung. Eine integrierte Lipo-Batterie ermöglicht den kabellosen Betrieb über einen längeren Zeitraum.

Aufgrund seiner kompakten Größe und der langen Batterielaufzeit eignet er sich besonders für IoT-Systeme.

In diesem Projekt dient der M5StickC als tragbare Steuerungs- und Überwachungseinheit. An das Modul werden Temperatur- und Lichtsensoren angeschlossen. Das Modul wird über WLAN mit dem Server verbunden. Auf dem integrierten Display werden die Daten der Sensoren sowie Verbindungszustände und Meldungen angezeigt.

3.1.4 BME680

Der BME680 ist ein kompakter Umweltsensor von Bosh, der mehrere Funktionen in einem Modul vereint. Das Modul ist mit einem Gassensor, einem Feuchtesensor, einem Luftdrucksensor und einem Temperatursensor ausgestattet. Aufgrund seiner kompakten Größe und hohen Genauigkeit wird das Modul häufig im Bereich der Umweltsensorik eingesetzt.

Mit Hilfe des eingebauten Gassensors und einer vom Hersteller speziell entwickelten Software kann die Luftqualität (IAQ - Eng. Index of Air Quality) gemessen werden. Grundsätzlich handelt es sich um eine Zahl von 0 bis 500, wobei 0 die beste und 500 die schlechteste Luftqualität darstellt. Eine detaillierte Beschreibung des Sensors sowie der Bestimmung der IAQ ist im Datenblatt zum Modul BME680 [9] unter Kapitel 1.2 zu finden. Die Tabelle 5 enthält die wichtigsten technischen Daten des Moduls.

Der BME680 verfügt über I2C- und SPI- Schnittstellen für die Kommunikation mit anderen Geräten.

Modell	BME680
Versorgungsspannung	3,3 V
Schnittstellen	I2C, SPI
Temperaturbereich	-40 bis +85 °C
Luftfeuchtigkeitsbereich	0 – 100 %
Druckbereich	300 – 1100 hPa
Luftqualität	0 - Gute Qualität 500 - schlechte Qualität

Tabelle 5: Technische Daten des BME680

In meinem Projekt wird das Modul BME680 zur Messung von Temperatur und Luftqualität eingesetzt.

3.1.5 VCNL4040

Der VCNL4040 ist ein hochpräzises optisches Modul. Das Modul enthält einen Näherungssensor (Proximity Sensor PR), einen Lichtsensor (Ambient Light Sensor) und eine Infrarot-Leuchtdiode (IRED).

Mit Hilfe des Näherungssensors und der IRED kann das Modul den Abstand zu einem Objekt bis zu einer Entfernung von 200 mm in Echtzeit messen. Der Lichtsensor misst die Helligkeit der Umgebung.

Das VCNL4040 verfügt über eine I2C-Schnittstelle zur Kommunikation mit anderen Geräten. Eine detaillierte Beschreibung des Moduls sowie die technischen Daten sind dem Datenblatt [10] zu entnehmen. In der Tabelle 6 sind die wichtigsten Daten aufgelistet.

Modell	VCNL4040
Messprinzip	Infrarot-Näherungssensor
Versorgungsspannung	3,3 V
Schnittstellen	I2C
Erfassungsbereich	0 – 200 mm
Maximale Abtastrate	20 Hz

Tabelle 6: Technische Daten des VCNL4040

In diesem Projekt wird das Modul als Lichtsensor zur Bestimmung der Umgebungshelligkeit eingesetzt. Außerdem wird es zur Näherungserkennung verwendet, um berührungslose Steuerungen zu ermöglichen.

3.1.6 Raspberry Pi

3.2 Kommunikationsprotokolle

3.2.1 HTTP

3.2.2 MQTT

3.2.3 UART

3.2.4 I2C

3.3 Software

3.3.1 PlatformIO

3.3.2 Python

3.3.3 FastAPI

3.3.4 SQLAlchemy

3.3.5 WebSocket

3.3.6 Web

TODOOO kurzer Überblick auf die Webentwicklung mit HTML/CSS und JavaScript

4 Einrichtung der Hardwarekomponenten

4.1 Arduino

TOODOO wird leicht geändert und erweitert.

Der Arduino mit den angeschlossenen Sensoren stellt das Sicherheitskomponente des Systems dar. Dieses Teilsystem erkennt die Gefahren von Gas und Feuer und alarmiert den Benutzer. Außerdem stellt der Arduino der gesicherte Zugang zu dem Haus durch die Eingabe einer PIN. Die Anschließung der Sensoren und Aktoren an Arduino ist in der PA1 [1] beschrieben.

Der Arduino kann nicht direkt mit einem WLAN verbunden werden, da er kein WLAN-Modul besitzt. Für diesen Zweck habe ich ein ESP8266-Modul verwendet. Der wird mit dem Arduino durch eine UART-Schnittstelle angeschlossen und mit dem WLAN verbunden. Die Kommunikation zwischen Arduino und ESP8266 erfolgt über die serielle Schnittstelle. Der Arduino gibt die entsprechenden Kommanden durch Serial an ESP8266 weiter und der ESP8266 führt die Befehle aus. Die detaillierte Beschreibung zur Verbindung von Arduino und ESP8266 ist in dem Bericht zu meiner Projektarbeit 2 [2] Kapitel 4.1 zu finden.

Der Arduino sendet und empfängt die MQTT-Nachrichten via angeschlossenen ESP8266. Ein Mal pro Sekunde sendet der Arduino eine MQTT-Nachricht in das Topic „data/*iID_i*“ in dem die Bereitschaft der angeschlossenen Sensoren geteilt wird. Bei der Erkennung einer Gefahr wird sofort der Alarm ausgelöst und eine MQTT-Nachricht in das Topic „alarm/*iID_i*“ mit dem entsprechenden Text gesendet.

4.2 Raspberry Pi

Der Raspberry Pi dient als zentraler Server des Systems. Er stellt einen WLAN-Zugriffspunkt zur Verfügung, über den sich die Geräte mit meinem System verbinden. Als Netzwerkzentrale gewährleistet er eine stabile und sichere Kommunikation zwischen den Komponenten, selbst in Umgebungen ohne Internetzugang.

Die Geräte kommunizieren über die MQTT-Protokoll, wobei der Raspberry Pi den MQTT-Broker auf Basis von **Mosquitto** hostet. Die Geräte senden ihre Daten in spezifische Topics. Die Clients abonnieren diese Topics und erhalten die von Geräten gesendet Daten. Diese Daten werden von einem asynchronen MQTT-Client verarbeitet.

Der Raspberry Pi hostet zudem den API-Webserver, der als Schnittstelle für die Interaktion mit dem System dient. Die API ist mit **FastAPI** implementiert und läuft auf dem ASGI-Server **Uvicorn**. **Uvicorn** ermöglicht eine effiziente, asynchrone Verarbeitung von Anfragen und sorgt für eine hohe Leistungsfähigkeit.

Die von den Geräten empfangenen Daten werden durch den asynchronen MQTT-Client verarbeitet und über **WebSocket** direkt an die API weitergeleitet. Dadurch wird eine Echtzeitübertragung an die Benutzeroberfläche ermöglicht, ohne dass die Daten zwischengespeichert oder erneut abgefragt werden müssen.

Der genaue Ablauf für die Einrichtung des Raspberry Pi ist in dem Bericht zu meiner Projektarbeit 2 [2], Kapitel 2, beschrieben.

Diese Architektur ermöglicht die effiziente und schnelle Interaktion mit dem System. Das System ist zwar nur im lokalen Netzwerk verfügbar. Da keine Cloud-Dienste erforderlich sind, bietet das System eine erhöhte Sicherheit und ist von der Internetverbindung unabhängig.

4.3 Einrichtung der ESP-Module

In diesem Projekt werden drei verschiedene ESP-Module verwendet: ESP8266, ESP32 und der auf dem ESP32 basierende M5Stick. Jedes Modul erfüllt spezifische Aufgaben, aber alle Module

müssen sowohl mit dem WLAN als auch mit dem MQTT-Broker verbunden sein. Daher ist die Implementierung der Funktion `setup()` für alle drei Module ähnlich aufgebaut.

Beim Start des ESP-Moduls wird die Setup-Funktion aufgerufen. Zunächst wird das Modul mit der Funktion `setup_esp` entsprechend eingerichtet. Je nach Modul umfasst diese Funktion verschiedene Anfahrtroutinen. Dazu gehört unter anderem die Initialisierung der seriellen oder I2C-Kommunikation, die Einrichtung der angeschlossenen Sensoren und so weiter.

Anschließend werden mit der Funktion `connect_wifi_mqtt` (Listing 1) die WLAN- und MQTT-Verbindungen aufgebaut. Zunächst wird versucht die WLAN-Daten aus dem EEPROM zu laden. Das EEPROM wird mit einer Größe von 128 Byte initialisiert. Dies ist notwendig, bevor Daten aus dem Speicher gelesen werden können. Es werden zwei Felder vom Typ `char` mit vordefinierten maximalen Längen zum Speichern der Daten aus dem EEPROM erstellt (Zeile 5-6). Der Name des WLAN-Netzes (SSID) wird ab Adresse 0 bis 31 des EEPROM ausgelesen und mit der Funktion `EEPROM.get(0, eeprom_ssid)` im Char-Feld `eeprom_ssid` gespeichert. Das Passwort wird ab Adresse 32 aus dem EEPROM ausgelesen und im Char-Feld `eeprom_password` gespeichert.

Anschließend wird mit der Boolean-Funktion `is_valid_string()` geprüft, ob die aus dem EEPROM abgelesenen Daten gültig sind. Die Funktion gibt den Wert `False` zurück, wenn ein Standard-EEPROM-Zeichen mit dem Wert `0xFF` gefunden wurde. Dies bedeutet, dass EEPROM noch leer ist und die WLAN-Daten noch gespeichert werden müssen.

```
1  void connect_wifi_mqtt()
2  {
3      EEPROM.begin(128);
4      char eeprom_ssid[MAX_SSID_LENGTH] = {0};
5      char eeprom_password[MAX_PASSWORD_LENGTH] = {0};
6      EEPROM.get(0, eeprom_ssid);
7      EEPROM.get(32, eeprom_password);
8      EEPROM.end();
9      // Daten gefunden
10     if (is_valid_string(eeprom_ssid, MAX_SSID_LENGTH) &&
11         is_valid_string(eeprom_password, MAX_PASSWORD_LENGTH))
12     {
13         // verbinden mit WLAN
14         if (wifi_connect(eeprom_ssid, eeprom_password))
15         {
16             i2cScan.begin(SDA2, SCL2, 400000);
17             mqtt_connect();
18         }
19         else // die WLAN-Verbindung is schiefgelaufen
20         { setup_ap(); }
21     }
22     else // keine WLAN-Daten gefunden
23     { setup_ap(); }
24 }
```

Listing 1: ESP: `connect_wifi_mqtt()`

Sind die gültigen Werte im EEPROM gespeichert, versucht das Modul, sich mit diesen Zugangsdaten mit dem WLAN zu verbinden (Zeile 13). Die Funktion `connect_wifi` schaltet das WLAN-Modul in den Modus `Station`, was die Verbindung mit anderen WLAN-Netzwerken erlaubt. Danach wird es versucht mit dem WLAN mit übergebenen Zugangsdaten zu verbinden. In der Variable `wifi_repeat` ist die Anzahl der Versuche zur Erstellung der Verbindung

gespeichert. Jede Sekunde wird den Status der Verbindung überprüft. Wenn die Verbindung erfolgreich erstellt wurde, gibt die Funktion `True` aus.

Falls die aus dem EEPROM ausgelesenen Daten ungültig sind oder keine erfolgreiche WLAN-Verbindung aufgebaut werden kann, wird die Funktion `setup_ap()` (Listing 3) aufgerufen. Diese Funktion stellt einen Webserver bereit. Das WLAN-Modul wird in den Modus „Access Point“ versetzt. Die Konfiguration des WLAN-Zugangspunktes erfolgt über die Befehle `WiFi.softAP()` und `WiFi.softAPConfig()`, wobei die Parameter des Access Points in Listing 2 definiert sind.

Der Webserver wird durch das Objekt `server` der Klasse `AsyncWebServer` zur Verfügung gestellt und mit dem Befehl `server.begin()` gestartet.

```
1 // Name des Netzwerks, entspricht DEVICE_ID
2 const char AP_SSID[] = DEVICE_ID;
3 // Passwort = setup + DEVICE_ID
4 const char AP_PASSWORD[] = "setupes32";
5 // Gleich fuer alle ESP-Module
6 IPAddress ap_ip(10, 0, 0, 1);
7 IPAddress ap_gateway(10, 0, 0, 1);
8 IPAddress ap_subnet(255, 255, 255, 0);
9 AsyncWebServer server(80);
```

Listing 2: ESP: Access Point

```
1 void setup_ap()
2 {
3     WiFi.mode(WIFI_AP);
4     WiFi.softAP(AP_SSID, AP_PASSWORD);
5     WiFi.softAPConfig(ap_ip, ap_gateway, ap_subnet);
6     server.on("/", HTTP_GET, [](AsyncWebServerRequest *request)
7         { request->send(200, "text/html", html_page); });
8
9     server.on("/save", HTTP_POST, [](AsyncWebServerRequest *request)
10        {
11            // GET Daten aus Input
12            String ssid = request->getParam("ssid", true)->value();
13            String password = request->getParam("password", true)->value
14                ();
15            if (ssid.length() > MAX_SSID_LENGTH - 1 || password.length()
16                > MAX_PASSWORD_LENGTH - 1)
17            {
18                ap_on = false;
19                return;
20            }
21            // String in char[]
22            char new_ssid[MAX_SSID_LENGTH] = {0};
23            strncpy(new_ssid, ssid.c_str(), MAX_SSID_LENGTH - 1);
24            char new_password[MAX_PASSWORD_LENGTH] = {0};
25            strncpy(new_password, password.c_str(), MAX_PASSWORD_LENGTH
26                - 1);
27
28            // in EEPROM speichern
29            EEPROM.begin(128);
30            EEPROM.put(0, new_ssid);
31            EEPROM.put(32, new_password);
32            EEPROM.commit();
```

```

30     EEPROM.end();
31
32     request->send(200, "text/html", "WiFi saved. Rebooting...");
33     delay(1000);
34     ESP.restart(); });
35     ap_on = true;
36     server.begin();
37 }

```

Listing 3: ESP: setup_ap

Die HTTP-Routen werden innerhalb der Funktion `server.on()` festgelegt. Beim Aufruf der Root-Route (`/`) wird eine HTTP-GET-Anfrage bearbeitet, die mit einer HTML-Seite beantwortet wird. Diese Seite enthält ein Formular mit Input-Felder zur Eingabe der WLAN-Zugangsdaten. Der Benutzer soll der Name des Netzwerks und das Passwort eingeben.

Wenn der Benutzer auf die Taste „Submit“ drückt, wird eine HTTP-POST-Anfrage an die Route `./save` gesendet. Die Zugangsdaten werden aus dem `request`-Objekt ausgelesen und auf ihre maximale Länge überprüft. Wenn die Daten gültig sind, werden sie im EEPROM gespeichert. Der EEPROM wird initialisiert, die neuen Zugangsdaten werden gespeichert, und die Änderungen mit `EEPROM.commit()` übernommen. Anschließend wird das ESP-Modul mit `ESP.restart()` neu gestartet, um die neuen WLAN-Daten zu verwenden.

Nach erfolgreicher Verbindung mit dem WLAN, wird die Funktion `mqtt_connect()` (Listing 4) aufgerufen, um die Verbindung zum MQTT-Broker zu erstellen. Zunächst werden die Topics zum Senden und Empfangen der Nachrichten mit der Funktion `set_topics` erstellt. Dabei werden die Topics mit `Geräte-ID` für jedes Modul individuell formuliert.

Der `mqttClient` ist ein Objekt der Klasse `PubSubClient`. Die Funktion `setServer()` erhält als Parameter die IP-Adresse des MQTT-Brokers sowie den Port, auf dem der Broker lauscht. Diese Parameter sind als Konstanten im Programmcode definiert. Die Funktion `callback` wird beim Empfang einer MQTT-Nachricht ausgeführt. Anschließend verbindet sich das Modul mit dem MQTT-Broker und abonniert das `SUBSCRIBE_TOPIC`.

```

1  void mqtt_connect()
2  {
3      set_topics();
4      mqttClient.setServer(WiFi.gatewayIP(), MQTT_PORT);
5      mqttClient.setCallback(callback);
6      // Verbinden mit dem MQTT-Broker
7      if (mqttClient.connect(DEVICE_ID))
8      {
9          mqttClient.subscribe(SUBSCRIBE_TOPIC);
10         Serial.println("MQTT CONNECTED");
11     }
12     else {delay(1000);}
13 }

```

Listing 4: ESP: mqtt_connect

Die Funktion `callback` wird beim Empfang einer MQTT-Nachricht ausgeführt. In dieser Funktion sind die Reaktionen auf den erhaltenen Nachrichten aus dem Topic `command/ID` festgelegt. Abhängig vom Gerät kann diese Funktion angepasst und erweitert werden.

Der Text der empfangenen MQTT-Nachricht wird als `payload` an die Funktion übergeben und zunächst im `String message` gespeichert. Dieser enthält die auszuführenden Befehle, die je nach Gerät unterschiedlich interpretiert werden können. Jedes Gerät muss jedoch auf das Kommando `handshake` reagieren.

Wenn eine Handshake-Nachricht empfangen wurde, muss das Gerät mit seinen Konfiguration antworten (Listing 5). Zunächst werden zwei Arrays definiert: eines für die relevanten Datenfelder und eines für die möglichen Befehle. Im Array „`data_fields`“ sind die Schlüssel gelistet, die das Modul in das Topic `data/ID` sendet. Im Array „`commands`“ sind die Befehle gelistet, die das Modul ausführen kann. Je nach Modul variiert der Inhalt. Diese Arrays werden in einem JSON-Dokument strukturiert, wobei die Datenfelder und Befehle jeweils als separate Arrays innerhalb des Dokuments abgelegt werden:

```
{
  "data_fields": [ "temperature", "humidity"],
  "commands": [ "light_turn_on", "light_turn_off"]
}
```

Anschließend wird das JSON-Dokument serialisiert und in eine Zeichenkette umgewandelt. Diese serialisierte Nachricht wird dann über die MQTT-Verbindung an das Topic „`handshake/ID`“ gesendet.

```
1 void callback(char *topic, byte *payload, unsigned int length)
2 {
3     if (message == "handshake")
4     {
5         const char *data_fields[] = {"temperature", "humidity"};
6         const char *commands[] = {"light_turn_on", "light_turn_off"};
7
8         char json[128];
9         JsonDocument data;
10        JsonArray json_df = data.createNestedArray("data_fields");
11        JsonArray json_commands = data.createNestedArray("commands");
12
13        for (const char* df: data_fields){ json_df.add(df); }
14        for (const char* a: commands){ json_actions.add(a); }
15
16        serializeJson(data, json, sizeof(json));
17        mqttClient.publish(HANDSHAKE_TOPIC, json);
18    }
19 }
```

Listing 5: ESP: mqtt.connect

Diese Setup-Konfiguration wird bei allen ESP-Modulen verwendet. Je nach Modul wird diese Funktion erweitert sein. Beispielsweise wird bei Modulen mit Display angezeigt, ob die Verbindung aufgebaut wurde.

4.3.1 ESP8266

Der ESP8266 wird für die Verbindung des Arduinos mit dem WLAN verwendet. Zu seinem Aufgaben gehört das Senden und Empfangen der MQTT-Nachrichten in entsprechenden Topics. Wie in meinem Bericht zur Projektarbeit 2 [2] Kapitel 4.1 beschrieben, ist der ESP8266 über eine UART-Schnittstelle mit dem Arduino verbunden.

Beim Start des Moduls wird die Funktion `setup()` (Listing ??) ausgeführt. Zusätzlich leuchtet der blaue LED während der Einrichtung des Moduls. Der Diode leuchtet, wenn es keine Spannung auf dem GPIO 1 fällt.

```
digitalWrite(1, LOW); // Diode einschalten
digitalWrite(1, HIGH); // Diode ausschalten
```

Nach der erfolgreichen Einrichtung und Verbindung mit dem WLAN sowie MQTT-Broker, schaltet der Diode aus. Bei jedem Pogrammdurchlauf wird die Funktion `readSerialData` (Listing ??) aufgerufen. Stehen die Daten in der seriellen Schnittstelle zur Verfügung, werden diese zunächst als String ausgelesen und gespeichert. Der String wird auf ihre maximale Länge geprüft. Anschließend muss der String in ein char-Feld konvertiert werden. Um das Topic und den Text der Nachrichten zuzuordnen, wird nach der Position von „:“ gesucht. Alle Zeichen rechts von „:“ gehören zum Text der Nachricht und werden in der Variablen „`message`“ gespeichert. Aus den Zeichen links von „:“ wird das Topic gebildet. Arduino überträgt nur die Namen der Haupttopics. Das ESP8266 erweitert das von Arduino übergebene Topic mit der ID des Gerätes und formuliert das `PUBLISH_TOPIC` (Zeile 22-32). Wenn das Topic aus irgendeinem Grund nicht korrekt formuliert werden kann, wird die Nachricht im Haupttopic gesendet. Damit ist sichergestellt, dass die Nachricht versendet wird. Sobald das ESP8266 die Daten von dem Arduino empfängt, wird die MQTT-Nachricht an den MQTT-Broker mit dem Befehl „`mqttClient.publish(topic, message)`“ versandt.

Die Verarbeitung der empfangenen Nachricht erfolgt in der Funktion `callback` (Listing 5). Der ESP8266 abonniert nur das Topic, das in der Variablen `SUBSCRIBE_TOPIC` definiert ist. Beim Empfang der Nachricht aus dem Topic wird geprüft, ob die Nachricht aus dem dem Gerät zugewiesenen Topic stammt. Wenn dies der Fall ist, wird der Text der Nachricht über Serial an Arduino gesendet.

4.3.2 M5Stick

Der M5Stick wird zur Überwachung der Licht- und Luftbedingungen eingesetzt. Die Idee ist, dass der Sensor die aktuelle Wetterbedingungen überwacht und diese an dem Server sendet. Für diesen Zweck soll der Sensor an dem Fenster draußen positioniert werden.

Am Modul sind die Sensoren BME680 und VCNL4040 angebunden. Der BME680 erfasst Umweltdaten: Temperatur, Luftfeuchtigkeit und Gaswiderstand. Der VCNL4040 dient als Lichtsensor und misst Lichtbedingungen: Umgebungshelligkeit (Ambient Light), weißes Licht sowie Annäherung (Proximity). Diese Daten werden an dem Server via MQTT-Nachrichten gesendet.

Für die Verwendung der beiden Sensoren im Programmcode müssen die entsprechenden Bibliotheken importiert werden und die Klassenobjekten für die Sensoren erstellt werden (Listing 6).

```
1  #include <Adafruit_BME680.h>
2  #include <Adafruit_VCNL4040.h>
3  Adafruit_BME680 bme_sensor;
4  Adafruit_VCNL4040 vcnl4040 = Adafruit_VCNL4040();
```

Listing 6: M5Stick: Sensoren

Beim Start des Moduls wird die Funktion `setup()` aufgerufen. Zusätzlich zu der im Listing ?? durchgeführte Einrichtungen müssen noch die beiden Sensoren eingerichtet und im Betriebszustand gebracht werden. Dacher werden im Setup die Funktionen `vcnl_setup()` und `bme_setup()` für die Einrichtung der Sensoren aufgerufen.

Listing 7: M5Stick: vcnl_setup

4.3.3 ESP32

5 Softwareentwicklung

In diesem Abschnitt wird die Aufbau und Entwicklung der Software für das System beschrieben. Für die Entwicklung wurden Programmiersprachen Python und JavaScript mit mehreren Bibliotheken sowie die Markierungssprache HTML und CSS für die Visualisierung der Webseite verwendet.

Das Programm läuft auf dem Raspberry Pi. Der Programmcode ist in dem Projektordner unter `bachelor/code/spa` zu finden. Das Programm ist modular aufgebaut, wobei jedes Paket (Package) in einem eigenen Unterordner organisiert ist und eine spezifische Aufgabe erfüllt. Die Struktur des Programms sieht wie folgt aus:

```
bachelor/code/spa/  
|  
|--- app/  
|   |--- config/  
|   |--- db/  
|   |--- mqtt/  
|   |--- webserver/  
|   |--- __init__.py  
|   |--- __main__.py  
|  
|--- requirements.txt  
|--- tailwind.config.js
```

- **app/** — Hauptmodul des Programms. Enthält die Startdatei `__main__.py` sowie eine Initialisierungsdatei `__init__.py`, um das gesamte Modul als Package erkennbar zu machen.
- **config/** — Enthält die Einstellungen für die Datenbankverbindungen, MQTT-Client, Logger und den Webserver.
- **db/** — Verwaltet der Datenbank. Enthält die Funktionen für die Verbindung zur Datenbank, die Erstellung von Tabellen und die Interaktion mit der Datenbank.
- **mqtt/** — Enthält die Logik für die Kommunikation über das MQTT-Protokoll. Enthält Code für den Empfang, die Verarbeitung und das Senden von MQTT-Nachrichten.
- **webserver/** — Stellt die API-Endpunkte und eine Webseite zur Verfügung. Enthält Klassen zur Verwaltung der Benutzerinteraktionen und zur Bereitstellung der Webseite.

Zuerst müssen die benötigten Module auf dem Server installiert werden. Die Module sind in der Datei `requirements.txt` aufgelistet. Mit dem Befehl werden die Module heruntergeladen und installiert:

```
pip install -r requirements.txt
```

5.1 Datenbank

Struktur der Datenbank

Die Datenbank für dieses Projekt wurde mit SQLAlchemy implementiert. Ziel war es, eine relationale Datenbank zu erstellen, die modular und flexibel aufgebaut ist, sowie die Möglichkeit bietet die Datenbankoperationen ohne direkten SQL-Abfragen durchzuführen.

Die Funktionen zur Verwaltung der Datenbank sind in einem Paket zusammengefasst. Dieses Paket bündelt alle relevanten Komponenten, wie die Konfiguration der Datenbankverbindung, das Session-Management sowie die Definition der Datenbankmodelle. Durch diese modulare Struktur kann das Datenbankpaket in anderen Klassen oder Paketen problemlos importiert werden, um eine zentrale und einheitliche Interaktion mit der Datenbank zu ermöglichen.

Die Struktur des Packets sieht wie folgt aus:

```

bachelor/code/spa/db/
|
|--- __init__.py
|--- base.py
|--- database.db
|--- models.py
|--- queries.py

```

Die Datei `database.db` stellt die Datenbank dar. Es enthält alle Tabellen und Daten, die eingetragen wurden. Die Abbildung 1 zeigt die in der Datenbank gelegte Tabellen und deren Verknüpfungen:

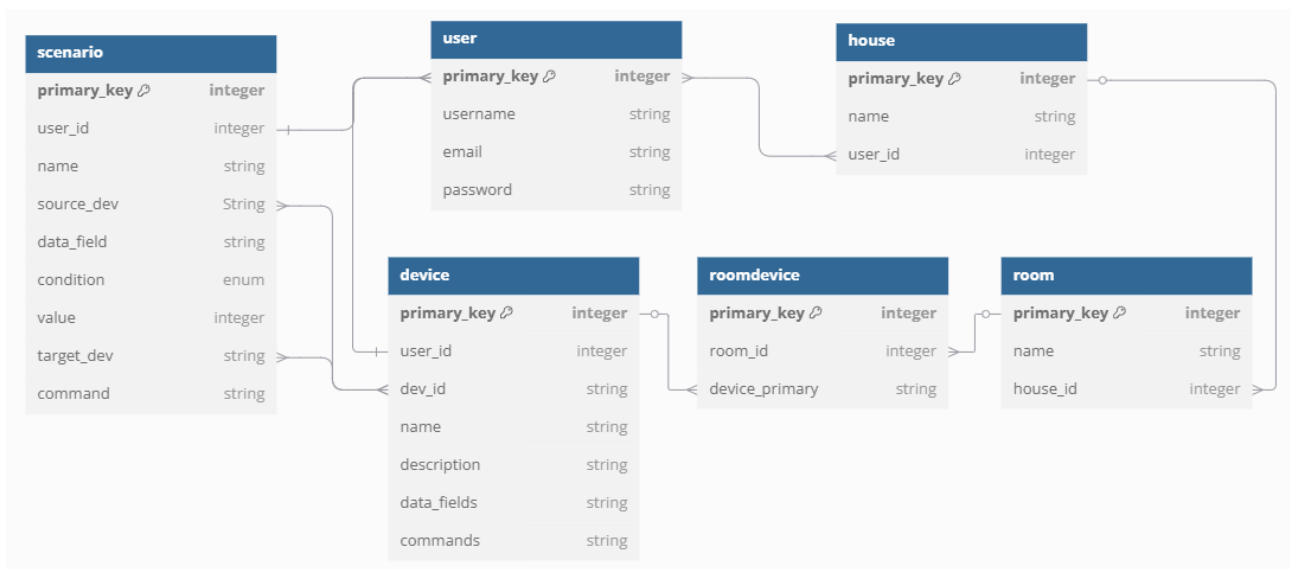


Abbildung 1: Datenbank

- **user** — Enthält die Benutzerdaten.
 - **primary_key**: Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
 - **username**: Der Benutzername, der für jeden Benutzer eindeutig ist.
 - **email**: Die E-Mail-Adresse des Benutzers, ebenfalls einzigartig.
 - **password**: Das Passwort des Benutzers, das verschlüsselt gespeichert wird.

Ein Benutzer kann mehrere Häuser und Geräte haben.

- **house** — Speichert die vom Benutzer erstellten Häuser.
 - **primary_key**: Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.

- **name:** Der vom Benutzer erstellte Name des Hauses.
- **user_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **user** verweist, um anzugeben, welchem Benutzer das Haus gehört.

Ein Haus gehört zu einem bestimmten Benutzer. Ein Haus kann mehrere Räume enthalten.

- **room** — Speichert die vom Benutzer erstellten Räume.
 - **primary_key:** Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
 - **name:** Der vom Benutzer erstellte Name des Raums.
 - **house_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **house** verweist.

Ein Raum gehört immer zu einem bestimmten Haus. Ein Raum kann mehrere Geräte enthalten.

- **device** — Speichert die vom Benutzer angemeldeten Geräte.
 - **primary_key:** Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
 - **dev_id:** Die eindeutige, vordefinierte ID des Geräts.
 - **name:** Der vom Benutzer erstellte Name des Geräts.
 - **user_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **user** verweist und angibt, welchem Benutzer das Gerät gehört.
 - **description:** Eine optionale Beschreibung des Geräts.
 - **data_fields:** Die Datenfelder, die vom Gerät geliefert und dem Benutzer zur Verfügung gestellt werden. Werden im CSV-Format gespeichert, getrennt durch „，“.
 - **commands:** Die Befehle, die das Gerät ausführen kann. Werden im CSV-Format gespeichert, getrennt durch „，“.

Ein Gerät gehört genau einem Benutzer und einem Raum. Ein anderes Gerät mit derselben Geräte-ID kann jedoch von einem anderen Benutzer angemeldet und einem anderen Raum zugeordnet werden.

- **roomdevice** — Speichert die Zuordnung der Geräte zu den Räumen.
 - **primary_key:** Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
 - **room_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **room** verweist.
 - **device_primary:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **device** verweist.
- **scenario:** — Speichert die vom Benutzer erstellten Szenarien.
 - **primary_key:** Ein Primärschlüssel vom Typ Integer, der automatisch inkrementiert wird.
 - **user_id:** Ein Fremdschlüssel, der auf den Primärschlüssel der Tabelle **user** verweist.

- **name:** Der vom Benutzer erstellte Name des Szenarios
- **source_dev:** Gerät, das das Szenario auslöst. Ein Fremdschlüssel, der auf der Spalte `dev_id` der Tabelle `device` verweist.
- **data_field:** Datenfeld des Quellgeräts.
- **condition:** Bedingung für das Auslösen der Szenario. Mögliche Werte sind „>“, „>“ und „==“, die die Bedingungen „größer als“, „kleiner als“ und „gleich“ repräsentieren.
- **value:** Der Wert des Datenfeldes.
- **target_dev:** Zielgerät, der ein Befehl beim Auslösen des Szenarios ausführt.
- **command:** Der Befehl, der das Zielgerät ausführt.

Ein Benutzer kann mehrere Szenarios erstellen.

Aufbau mit SQLAlchemy

Jetzt müssen diese Tabellen und deren Verknüpfungen in Form der Modellen in SQLAlchemy definiert werden. Die Models sind in der Datei `models.py` (Listing ??) definiert.

Zunächst müssen die benötigte Module importiert werden. Das Modul `bcrypt` wird für die Entschlüsselung und Verifizierung der Passwörter verwendet. Die Module aus der Bibliothek `sqlalchemy` enthalten die Funktion für die Interaktionen mit der Datenbank. Das Modul `.base` ist die Datei `base.py` (Listing 8). Da wird nur ein `declarative_base` importiert, und in der Variable `Base` initialisiert.

```
1 from sqlalchemy.orm import declarative_base
2 Base = declarative_base()
```

Listing 8: db: base.py

Die Tabellenmodelle werden in Klassen definiert, die von der `Base` erben. Dies ist die Standardeinstellung für SQLAlchemy. Die Variable `__tablename__` legt der Name der Tabelle fest. Die Spalten sind als Objekte der Klasse `Column` mit gegebenen Parametern definiert. Die Klasse `UserModel` enthält eine Funktion für die Validierung der Passwörter (Zeile 17). Die Funktion bekommt ein Password-String als Parameter und vergleicht dieser mit dem im Modell gespeicherten Passwort.

In SQLAlchemy, wenn ein `ChildModel` Modell einen Fremdschlüssel des `ParentModel` Modells beinhaltet, muss eine Beziehung mit dem `ChildModel` Modell in der `ParentModel` Klasse definiert werden. Dies wird verwendet, um Datenbankkonflikte beim Löschen oder Aktualisieren von Einträgen im Modell `ChildModel` zu vermeiden.

Die Klasse `HouseModel` enthält neben der Definition der Spalten eine Beziehung zwischen `HouseModel` und `RoomModel` im Attribut `rooms`. Die Beziehung ist so definiert, dass alle zugehörigen Räume gelöscht werden, wenn das Haus gelöscht wird. Außerdem ist in der Klassenvariablen `__table_args__` mit dem Befehl `UniqueConstraint('name', 'user_id')` (Zeile 27) definiert, dass die Kombination der Spalten `name` und `user_id` eindeutig sein muss. Ein Benutzer kann also nicht 2 Häuser mit dem gleichen Namen anlegen.

Die Klassen `RoomModel` und `DeviceModel` haben eine Beziehung zur Klasse `RoomDeviceModel`, und in der Klasse `RoomDeviceModel` werden die Beziehungen zu den beiden Klassen definiert. Damit wird sichergestellt, dass beim Löschen eines Raumes oder eines Gerätes auch die entsprechende Verknüpfung gelöscht wird. Dabei ist zu beachten, dass beim Löschen eines Raumes das zum Raum gehörende Gerät nicht gelöscht wird und umgekehrt.

Interaktionen mit der Datenbank

In der Datei `queries.py` (Listing ??) sind die benötigten Funktionen für die Interaktion mit der Datenbank definiert. Diese Funktionen ermöglichen das Einfügen, Löschen, Abrufen und Aktualisieren der Daten in der Datenbank.

Jede Funktion bekommt ein `AsyncSession` als erster Parameter. Über diese Session werden die Operationen in der Datenbank durchgeführt. Das erlaubt auch asynchroner Zugriff zur Datenbank.

Als Beispiel werde ich die Struktur der Funktion `add_user()` näher erläutern. Diese Funktion fügt einen neuen Benutzer in die Datenbank ein. Die Funktion erhält als Parameter `AsyncSession` und die Benutzerdaten, d.h. Benutzername, Email und Passwort. Zuerst wird das Passwort verschlüsselt. Anschließend wird ein Objekt `new_user` der Klasse `UserModel` mit den übergebenen Parametern erzeugt. Dieses Objekt wird mit dem Befehl `db_session.add(new_user)` in die Datenbank eingefügt. Nach dem Einfügen muss der Befehl `db_session.commit()` aufgerufen werden, um die Änderungen in der Datenbank zu speichern. Anschließend wird das `new_user` aktualisiert und der Primärschlüssel zurückgegeben.

Beim Arbeiten mit der Datenbank müssen mögliche Fehler behandelt werden. Deshalb enthält jede Funktion einen `Try-Catch-Block`.

Ein `IntegrityError` wird ausgelöst, wenn eine Datenbankoperation gegen eine Integritätsbedingung der Datenbank verletzt. Er wird bei Verletzung einer Fremdschlüsselbeziehung, der Unique Constraints oder beim Einfügen von Daten des falschen Typs erzeugt.

Ein `SQLAlchemyError` umfasst alle Fehler, die von SQLAlchemy ausgelöst werden, wenn die Operation nicht erfolgreich war. Dieser Fehler wird bei Syntaxfehlern in der SQL-Abfrage oder bei Fehlern in der Datenbankverbindung ausgelöst.

Alle anderen möglichen Fehler, die keine `SQLAlchemy`-Fehler sind, werden im Block `Exception` gesammelt.

Bei jedem Fehler wird der Fehler geloggt und die Session zurückgesetzt, damit keine unvollständigen Daten in der Datenbank gespeichert werden. Anschließend wird eine `HTTPException` mit Status und Beschreibung des Fehlers zurückgegeben.

Initialisierung der Datenbank

Beim Import des Moduls `db` wird die Klasse `__init__` (Listing ??) aufgerufen. In dieser Klasse wird zunächst der Logger initialisiert. Anschließend wird eine asynchrone Engine `async_engine` für die Verbindung zur Datenbank erzeugt. Mit dem Parameter `async_engine` wird das Objekt `async_session` der Klasse `Sessionmaker` erzeugt. Die Funktion `create_tables()` initialisiert alle Tabellen in den Klassenmodellen, die von der Klasse `Base` erben. Die Funktion `get_session()` gibt eine asynchrone Session aus der `async_session` zurück. Mit dieser Session wird dann auf die Datenbank zugegriffen.

5.2 Webbasierte Schnittstelle (API)

Um die Benutzerinteraktionen mit dem System zu ermöglichen, muss eine Schnittstelle gebaut werden. Diese Schnittstelle muss den gesicherten Zugriff zu dem System bieten, die notwendigen Informationen bereitstellen, auf die Benutzerinteraktionen reagieren und die Daten verwalten. Nach sorgfältiger Abwägung verschiedener Optionen habe ich mich für die Implementierung einer webbasierten Schnittstelle auf Basis von `FastAPI` entschieden.

Ein entscheidender Vorteil von `FastAPI` gegenüber anderen Frameworks wie `Flask` oder `Django` in diesem Projekt ist die native Unterstützung asynchroner Programmierung. Dadurch kann `FastAPI` Anfragen effizient bearbeiten, indem sie mehrere Tasks gleichzeitig ausführt, anstatt blockierend auf das Ende einzelner Prozesse zu warten. Dies ist für mein System besonders vorteilhaft, da mehrere Benutzerinteraktionen, Datenverarbeitung und auch MQTT-

Kommunikation parallel ablaufen müssen. Darüber hinaus bietet **FastAPI** automatisch generierte Dokumentation für die API.

Die zentrale Idee hinter der Entwicklung der API besteht darin, dem Benutzer eine effiziente und sichere Möglichkeit zu bieten, IoT-Geräte in einer strukturierten Umgebung zu verwalten. Dies umfasst das Hinzufügen neuer Geräte, die Gruppierung dieser Geräte in Räumen sowie die Verwaltung mehrerer Räume innerhalb eines Hauses. Darüber hinaus ermöglicht die API die kontinuierliche Überwachung und Steuerung der hinzugefügten Geräte.

Der Webserver ist, wie auch anderen Komponenten, in einem Paket namens **webserver** strukturiert. Die Struktur des Packets ist wie folgt aufgebaut:

```
bachelor/code/spa/webserver
|
|--- __init__.py
|--- routes.py
|--- services.py
|--- templates/
|--- static/
```

Die Kommunikation zwischen Client und Server erfolgt über HTTP-Methoden:

- **GET:** Zum Abrufen von Informationen aus dem System.
- **POST:** Zum Übermitteln neuer Daten oder Benutzerinteraktionen.
- **PUT:** Zur Aktualisierung vorhandener Informationen.
- **DELETE:** Zum Löschen von Daten.

Im Skript **routes.py** sind die API-Endpunkte definiert, die verschiedene Funktionen des Systems bereitstellen. Jeder Endpunkt ist dabei einer bestimmten Aufgabe zugeordnet.

Zur besseren Strukturierung und Modularisierung des Codes wurden die Hilfsfunktionen in das Skript **services.py** ausgelagert. Diese Funktionen enthalten die Anwendungslogik, verarbeiten die Daten und stellen sie zur Verfügung, verwalten die Verbindungen zum WebSocket.

Nach dem Import der erforderlichen Module wird ein Router mit **APIRouter()** erstellt, um die API-Endpunkte zu definieren. Der Router wird anschließend mit der Methode **include_router()** in das Hauptobjekt der FastAPI-Anwendung integriert.

Um auf Anfragen mit HTML-Dateien zu antworten, wird ein **Jinja2Templates**-Objekt erstellt. Dabei wird der Pfad zum Ordner **templates/** angegeben, der die HTML-Vorlagen enthält. Dieses Objekt ermöglicht es, dynamische HTML-Antworten zu generieren, indem Daten aus den Endpunkten in die Vorlagen eingebunden werden.

Jede Funktion, die auf die Datenbank zugreift, benötigt als Parameter eine Instanz der Klasse **AsyncSession**. Im Modul **routes.py** wird dazu die Funktion **get_session()** aus dem Paket **db** verwendet, die für die Erstellung der Datenbanksitzungen zuständig ist. Durch die Verwendung des Abhängigkeitsmechanismus **Depends(get_session)** wird bei jedem Aufruf einer Router-Funktion automatisch eine neue Datenbanksitzung erzeugt und an die entsprechenden Unterfunktionen weitergegeben. Dies ermöglicht eine effiziente und asynchrone Kommunikation mit der Datenbank, wodurch mehrere Anfragen gleichzeitig ohne blockierende Wartezeiten bearbeitet werden können.

Durch die Verwendung von FastAPI-Routern und einer klar strukturierten Modulanordnung bleibt die API übersichtlich, leicht wartbar und erweiterbar. Die Trennung von Präsentationslogik (HTML, CSS) und Daten (API) wird ebenfalls gewährleistet.

5.2.1 API-Endpunkte

TODO Eine Liste von allen API-Endpunkten mit Beschreibung.

5.2.2 Sicherheit

Sicherheit spielt bei diesem System eine wichtige Rolle. Für die API gibt es zwei kritische Sicherheitspunkte: die Übertragung sensibler Daten und die Authentifizierung der Benutzer.

Um sensible Daten, wie Passwörter, sicher zu speichern, wird der Algorithmus `bcrypt` verwendet. Anstatt Passwörter im Klartext zu speichern, wird bei der Registrierung eines Benutzers das Passwort gehasht. Der Hash wird in der Datenbank gespeichert, nicht das Passwort selbst. Bei der Anmeldung wird das eingegebene Passwort mit dem gespeicherten Hash verglichen, ohne dass das Originalpasswort jemals entschlüsselt wird. Dadurch sind die Benutzerdaten auch dann geschützt, wenn die Datenbank kompromittiert wird.

Zur Authentifizierung und Identifizierung des Benutzers nach erfolgreicher Anmeldung wird das JWT-Verfahren³ verwendet. Die Funktion `services.create_jwt_token()` generiert ein JWT-Token, das an den Benutzer zurückgegeben wird. Dieses Token enthält die Benutzer-ID in verschlüsselter Form, besitzt eine in der Konfiguration definierte Ablaufzeit (`Config.JWT_EXPIRE_TIME`) und wird mit einem geheimen Schlüssel (`Config.JWT_SECRET_KEY`) signiert.

Das Token muss clientseitig gespeichert und bei jeder Anfrage im HTTP-Header mitgesendet werden, um die Authentifizierung des Benutzers sicherzustellen. Der entsprechende Header muss dabei die folgende Struktur aufweisen:

```
headers: {  
    "auth": "Bearer <JWT TOKEN>"  
}
```

Das Request-Objekt wird an die Funktion `routes.get_token()` übergeben, die das JWT-Token aus dem HTTP-Header extrahiert. Dabei wird geprüft, ob der Header das Feld „auth“ enthält und ob dessen Wert mit dem Schlüsselwort „Bearer“ beginnt. Ist dies nicht der Fall, wird eine Exception ausgelöst, da das Token in einem ungültigen Format vorliegt. Im normalen Ablauf wird das Token aus dem Header extrahiert und zur weiteren Verarbeitung zurückgegeben.

Die Verifizierung des Tokens stellt eine grundlegende Voraussetzung für alle API-Funktionen dar. Nur gültige und nicht abgelaufene Tokens ermöglichen den Zugriff auf geschützte Ressourcen der API.

Nach der Extraktion wird das Token als Parameter an die Funktion `services.verify_token()` übergeben, um seine Gültigkeit zu überprüfen. Zunächst wird das Token entschlüsselt, und die darin enthaltene Benutzer-ID wird extrahiert. Ist das Token abgelaufen, ungültig, fehlerhaft oder fehlen notwendige Informationen, wird eine entsprechende Fehlermeldung ausgegeben und eine HTTP-Exception mit dem entsprechenden Statuscode ausgelöst. Andernfalls wird die Benutzerkennung für nachfolgende Verarbeitungsschritte zur Verfügung gestellt.

Anmeldung

Damit ein Benutzer die Funktionen der API nutzen kann, muss zunächst ein Konto erstellt werden. Dies erfolgt durch Senden eines POST-Requests an die Route `/sign_up`. Der Request muss die erforderlichen Parameter `username`, `email` und `password` enthalten. Nach Erhalt der Anfrage wird die Funktion `signup_post()` aufgerufen, die die eingegebenen Daten verarbeitet und die Benutzerregistrierung durchführt.

Die Registrierungsdaten werden als `SignUpModel`-Objekt entgegengenommen und an die Funktion `services.signup_user()` weitergeleitet. Diese Funktion übernimmt die Validierung

³JSON Web Token

der Eingaben, indem sie prüft, ob alle erforderlichen Felder ausgefüllt wurden. Wenn ein Wert fehlt, wird die entsprechende Fehlermeldung geloggt und zurückgegeben. Sind alle Daten vollständig, wird der Benutzer mit der Funktion `queries.add_user()` in der Datenbank gespeichert. Diese Funktion gibt ein Fehler zurück, wenn der Benutzername oder die Email schon in der Datenbank gespeichert sind.

Nach erfolgreicher Speicherung wird ein JWT-Token generiert und zurückgegeben, das die Benutzer-ID enthält. Mit diesem Token kann der Benutzer direkt angemeldet werden. Tritt während des Prozesses eine Exception auf, so wird diese geloggt und eine entsprechende Fehlermeldung ausgegeben.

Einloggen Um einem bereits registrierten Benutzer den Zugriff auf die API zu ermöglichen, ist eine Authentifizierung erforderlich. Diese erfolgt durch das Senden eines POST-Requests an die Route `/login`. Der Request muss die Parameter `username` und `password` enthalten, die im `user_data`-Objekt übergeben werden. Nach Erhalt der Anfrage wird die Funktion `login_post()` aufgerufen, die die Authentifizierung des Benutzers vornimmt.

Die Benutzerdaten werden als `UserLoginModel` entgegengenommen und an die Funktion `services.auth.user()` übergeben. Diese Funktion überprüft, ob die erforderlichen Felder `username` und `password` ausgefüllt wurden. Ist es nicht der Fall, wird eine entsprechende Fehlermeldung ausgegeben. Anschließend wird mit der Funktion `queries.get_user_data()` geprüft, ob ein Benutzer mit dem angegebenen `username` in der Datenbank existiert. Wird kein entsprechender Datensatz gefunden, wird eine `HTTPException` mit dem Statuscode 401 und der Meldung „Invalid username“ ausgelöst.

Wenn der Benutzer existiert, wird das eingegebene Passwort mit der gespeicherten verschlüsselten Version verglichen. Ist die Prüfung erfolgreich, gibt die Funktion den Primärschlüssel zurück. Andernfalls wird die Fehlermeldung „Wrong password“ zurückgegeben.

Nach erfolgreicher Authentifizierung wird in der Funktion `login_post()` ein JWT-Token erzeugt und zurückgegeben, das die Benutzerkennung enthält. Tritt während des Prozesses eine Exception auf, wird diese protokolliert und eine entsprechende Fehlermeldung ausgegeben.

Nach dem erfolgreichen Einloggen und Erhalt des Tokens, kann der Benutzer zu den Funktionen der API zugreifen.

5.2.3 Aufbau

Haus und Raum

Der Benutzer kann seine Geräte nach Räumen und die Räume nach Häusern gruppieren. Dies erleichtert die Verwaltung und Überwachung der Geräte. Besonders im Frontend ist dies sehr nützlich, da es eine klare Strukturierung des Hauses geschaffen wird. Dabei kann ein Benutzer mehrere Häuser besitzen.

Um ein Haus zu anzulegen, muss ein POST-Request an die Route `add_house` gesendet werden. Der Request muss im Header ein Bearer-Token und im Body den Namen des Hauses enthalten. Nach Erhalt des Requests wird die Funktion `add_house_post()` aufgerufen. Standardmäßig wird das Token validiert, und die Benutzer-ID wird aus dem Token extrahiert, um das Haus dem entsprechenden Benutzer zuzuordnen. Wenn eine gültige Benutzer-ID aus dem Token erzeugt wurde, wird diese zusammen mit der Instanz der Klasse `AsyncSession` und dem Namen des Hauses an die Funktion `services.create_new_house()` übergeben.

Die Funktion `services.create_new_house()` prüft zunächst, ob ein gültiger Hausname übergeben wurde. Anschließend wird mit der Funktion `queries.add_new_house()` ein neuer Eintrag in der Datenbank angelegt und dem Benutzer zugeordnet. Wenn die Erstellung erfolgreich war, gibt die Funktion eine Bestätigung zurück. Andernfalls oder im Fehlerfall wird eine entsprechende Fehlermeldung generiert und geloggt.

Das Verfahren zum Hinzufügen eines Raumes zu einem House ist im Wesentlichen dasselbe wie bei der Erstellung eines House. An die Route `add_room` wird ein `POST`-Request mit dem Token im Header und mit dem Namen des Raumes sowie der zugehörigen House-ID als `RoomModel` im Inhalt gesendet. Auch hier wird das JWT-Token standardmäßig validiert, und die Benutzer-ID daraus extrahiert.

Die Daten des Raumes und die Benutzer-ID werden an die Funktion `services.add_room()` übergeben. Diese Funktion holt zunächst mit dem Befehl `queries.get_house()` die Daten des Hauses aus der Datenbank, um den Besitzer des Hauses zu bestätigen. Stimmt die übergebene Benutzer-ID mit der in der Datenbank gespeicherten Benutzer-ID des Hausbesitzers überein, wird mit der Funktion `queries.add_new_room()` ein neuer Eintrag in der Datenbank erstellt. Im Erfolgsfall wird eine Bestätigung zurückgegeben.

Wenn der Benutzer nicht berechtigt ist, einen Raum zu dem angegebenen Haus hinzuzufügen, wird eine Meldung über unberechtigten Zugriff geloggt und als Fehlerantwort zurückgegeben. Das Löschen eines Hauses oder Raumes erfolgt ähnlich wie das Hinzufügen: Es wird ein `POST`-Request an die entsprechenden Routen (`/delete_house` bzw. `delete_room`) gesendet, wobei im Header das Bearer-Token und im Inhalt des Requests die zu löschende Haus- bzw. Raum-ID übermittelt wird. Dabei ist zu beachten, dass beim Löschen eines Hauses auch alle zugehörigen Räume gelöscht werden. Dies wird durch die Beziehung `rooms` in `HouseModel` in der Datenbank definiert.

Beim Löschen eines Hauses wird ein Request mit den Benutzer- und House-ID an die Route `/delete_house` gesendet. Nach der Validierung des Tokens und Erzeugung der Benutzer-ID werden die Daten an die Funktion `services.delete_house()` übergeben. Diese Funktion prüft, ob das Haus dem Benutzer gehört und löscht mit dem Befehl `queries.delete_house()` den zugehörigen Eintrag aus der Datenbank. Die zugehörigen Räume werden ebenfalls gelöscht. Im Fehlerfall wird der entsprechende Fehler geloggt und ausgegeben.

Das Löschen eines Raumes ist ähnlich dem Löschen eines Hauses. Ein Request mit der Raum- und Haus-ID wird an die Route `delete_room` gesendet. Nach der Validierung des Tokens und Erzeugung der Benutzer-ID werden die Daten an die Funktion `services.delete_room()` übergeben. Wenn der Benutzer der Eigentümer des Hauses mit der übergebenen House-ID ist, wird der Raum aus der Datenbank gelöscht. Tritt ein Fehler auf, wird dieser geloggt und die entsprechende Meldung wird ausgegeben.

Die Route `/get_houses` gibt alle Häuser des Benutzers im JSON-Format zurück. Der Request muss im Header nur das Token enthalten, aus dem die Benutzer-ID erzeugt wird. Die Benutzer-ID wird an die Funktion `services.get_houses()` übergeben, die Benutzer-ID an die Funktion `queries.get_houses_on_user()` übergibt. Diese Funktion gibt eine Liste von `HouseModel` zurück, die dem Benutzer gehören. Diese Liste wird in eine „List of Dictionaries“ umgewandelt, die alle Häuser, die zugehörigen Räume und die Geräte enthält. Anschließend wird ein `JSONResponse`-Objekt zurückgegeben.

Device

Die zentrale Aufgabe der API ist die Überwachung der angeschlossenen Geräte und die Benachrichtigung der Benutzer über erkannte Gefahren. Damit dieses System genutzt werden kann, muss der Benutzer die Möglichkeit haben, seine Geräte in das System einzubinden.

Jedes Gerät verfügt über eine eindeutige Geräte-ID, die im Programmcode des Gerätes fest hinterlegt ist und zur Identifikation innerhalb des Systems dient. Die Speicherung der Geräte erfolgt in der Datenbank innerhalb der Tabelle `device` in Form von `DeviceModel`-Objekten.

Die Integration eines neuen Gerätes erfolgt durch das Senden eines `POST`-Requests an die Route `/add_new_device`. Der Header des Requests muss ein gültiges JWT-Token enthalten, aus dem die Benutzer-ID ausgelesen wird. Im Body des Requests muss die eindeutige Geräte-ID angegeben werden. Optional kann ein vom Benutzer erstellter Name und eine Beschreibung

des Gerätes sowie die Raum-ID für die Zuordnung des Gerätes zum Raum übergeben werden. Der Name dient ausschließlich der Benutzerfreundlichkeit, indem er eine eindeutige Identifikation des Gerätes ermöglicht. Die übergebenen Daten werden innerhalb der API als Instanz `services.DeviceModel` entgegengenommen und verarbeitet.

Nach Erhalt des Requests wird die Funktion `add_new_device_post` aufgerufen. Zuerst wird das Token validiert und die Benutzer-ID aus dem Token erzeugt. Anschließend werden die Gerätedaten an die Funktion `services.add_new_device` übergeben. Diese prüft zunächst, ob sowohl die Geräte-ID als auch der Gerätenamen im Request enthalten sind. Fehlt eine dieser Angaben, wird der Vorgang abgebrochen und eine Fehlermeldung zurückgegeben.

Zusätzlich hat der Benutzer die Möglichkeit, das Gerät direkt einem bestimmten Raum innerhalb des Hauses zuzuordnen. Dazu muss im Body des Requests die entsprechende Raum-ID angegeben werden. Die Funktion `services.add_new_device` sucht in diesem Fall, mit dem Befehl `queries.get_house_by_room` nach dem entsprechenden Haus. Anschließend wird geprüft, ob die aus dem Token extrahierte Benutzer-ID mit der des Hauseigentümers übereinstimmt. Ist dies nicht der Fall, wird der Zugriff verweigert, der Vorgang abgebrochen und eine entsprechende Fehlermeldung zurückgegeben.

Anschließend wird mit dem Befehl `queries.add_new_device` das Gerät zunächst ohne direkte Raumzuordnung in der Datenbank gespeichert. Ist eine Raum-ID vorhanden, wird mit dem Befehl `queries.add_room_device` ein Eintrag in der Tabelle `roomdevice` erzeugt, der den Primärschlüssel des Gerätes mit der Raum-ID verknüpft.

Diese strukturierte Vorgehensweise gewährleistet nicht nur eine sichere und konsistente Registrierung neuer Geräte, sondern bietet dem Benutzer auch die Möglichkeit, seine Smart-Home-Geräte flexibel zu verwalten und eindeutig zu identifizieren.

5.3 MQTT-Client

Die Geräte senden kontinuierlich ihre Daten über die MQTT-Verbindung an den MQTT-Broker. Jedes Gerät nutzt sein separates Topic, das sich auf die Geräte-ID bezieht. Um die Daten eines bestimmten Geräts zu empfangen, muss ein MQTT-Client das entsprechende Topic abonnieren.

Der MQTT-Client für den Webserver ist im Paket `mqtt` enthalten. In der Datei `client.py` ist die Klasse `MQTTClient` implementiert, die eine asynchrone Schnittstelle zur Verarbeitung von MQTT-Nachrichten bereitstellt.

Die Implementierung basiert auf der Bibliothek `aiomqtt`. Durch die asynchrone Architektur kann der Client effizient auf die eingehenden MQTT-Nachrichten reagieren, ohne blockierende Operationen auszuführen.

Die asynchrone Funktion `start_client()` enthält eine Endlosschleife und erwartet als Parameter eine Liste von Topics, die der Client abonnieren soll. Diese Funktion wird beim Start des Webserver mit dem Befehl `asyncio.create_task()` als asynchrone Aufgabe gestartet. Dadurch wird sichergestellt, dass die MQTT-Verbindung parallel zu anderen asynchronen Aufgaben läuft. Zudem stellt die Endlosschleife sicher, dass die Verbindung nach einer Unterbrechung oder einem Fehler automatisch wiederhergestellt wird.

Innerhalb der asynchronen Schleife wird zunächst mit dem Befehl `async with aiomqtt.Client(...)` die Verbindung zu dem MQTT-Broker aufgebaut. Anschließend werden die übergebenen Topics abonniert. Danach empfängt der Client die MQTT-Nachrichten asynchron über die `async for`-Schleife, die auf `client.messages` lauscht. Jede eingehende Nachricht wird an die Funktion `process_message()` weitergeleitet, die für die weitere Verarbeitung verantwortlich ist.

Die Funktion `process_message()` extrahiert zunächst die Geräte-ID aus dem Topic. Anschließend wird der Payload der Nachricht dekodiert und in ein JSON-Objekt `data` umgewandelt. Diese Daten werden dann über WebSockets an den Webserver weitergeleitet. Dazu wird die Klasse `WebsocketHandler` aus der Datei `services.py` importiert, und die Daten werden an

die Funktion `send_data()` dieser Klasse übergeben.

Durch diese Architektur kann der Webserver in Echtzeit auf neue MQTT-Nachrichten reagieren und die empfangenen Daten effizient an verbundene Clients weiterleiten.

5.4 WebSocket

Die Geräte senden kontinuierlich Daten über die MQTT-Verbindung an den MQTT-Broker. Diese Daten müssen verarbeitet und dem Benutzer in Echtzeit angezeigt werden. Zu diesem Zweck wird eine WebSocket-Schnittstelle bereitgestellt, die eine direkte und effiziente bidirektionale Kommunikation ermöglicht.

Die Route `mqtt/device/<device_id>` stellt eine WebSocket-Verbindung zur Verfügung, wobei `device_id` für die eindeutige Geräte-ID steht. Die Klasse `WebsocketHandler` in der Datei `services.py` verwaltet sämtliche WebSocket-Verbindungen und deren Lebenszyklus. Das Feld `active_connections` speichert alle aktiven WebSocket-Verbindungen in Form einer „List of Dictionaries“, wobei jedes Dictionary eine bestehende Verbindung mit der zugehörigen `device_id` sowie dem WebSocket-Objekt enthält.

Die Autorisierung des Benutzers, der eine Verbindung zum WebSocket aufbauen möchte, erfolgt über die Funktion `auth_websocket()`. Diese akzeptiert zunächst die WebSocket-Verbindung und wartet asynchron auf die erste Nachricht. Die erste Nachricht, die an dem WebSocket gesendet wird, dient als Initialisierungsnachricht und muss ein Token erhalten, das ähnlich wie ein HTTP-Header aufgebaut ist:

`"auth": "Bearer <TOKEN>"`

Aus dem übergebenen Token wird die Benutzer-ID extrahiert. Anschließend wird mit dem Befehl `queries.verify_user_device(...)` geprüft, ob der authentifizierte Benutzer berechtigt ist, auf das angegebene Gerät zuzugreifen. Fehlt das Token oder besitzt der Benutzer keine Zugriffsrechte auf das Gerät, wird eine entsprechende Fehlermeldung ausgegeben und geloggt.

Nach erfolgreicher Autorisierung des Benutzers wird die Funktion `connect()` aus der Klasse `WebsocketHandler` aufgerufen. Diese fügt die neue Verbindung in die Liste von aktiven Verbindungen hinzu. Die Verbindung bleibt bestehen, bis sie entweder vom Benutzer geschlossen oder durch einen Netzwerkfehler unterbrochen wird.

Die Kommunikation über das WebSocket-Protokoll erfolgt in einer Endlosschleife, die ständig auf eingehende Nachrichten wartet. Wird die Verbindung vom Benutzer oder durch andere Ereignisse unterbrochen, wird diese mit der Funktion `disconnect` aus der Liste der aktiven Verbindungen entfernt.

Die Funktion `send_data()` ermöglicht dNas gezielte Senden von Daten über die WebSocket-Schnittstelle. Dabei wird geprüft, ob eine aktive WebSocket-Verbindung mit der entsprechenden `device_id` existiert. Falls ja, werden die zu sendenden Daten als JSON-Objekt über die WebSocket-Schnittstelle an das Gerät übertragen. Tritt ein Übertragungsfehler auf, wird die Verbindung geschlossen und aus der Liste der aktiven Verbindungen entfernt.

Die Implementierung dieser WebSocket-Schnittstelle ermöglicht eine effiziente und latenzarme Datenübertragung zwischen dem MQTT-Broker und den angebundenen Geräten.

5.5 Frontend

TODO Das Frontend der Webanwendung ist als Single Page Application (SPA⁴) implementiert. Eine SPA ist eine Webanwendung, die auf einer einzigen HTML-Seite geladen wird und

⁴Eng. Single Page Applicaiton

deren Inhalt dynamisch durch JavaScript aktualisiert wird. Dies führt zu einer verbesserten Benutzererfahrung, da der Wechsel zwischen verschiedenen Ansichten schneller erfolgt und weniger Serveranfragen notwendig sind.

Das Frontend der Anwendung besteht aus einer HTML-Seite zur Strukturierung der Inhalte, JavaScript für die interaktive Funktionalität sowie CSS-Dateien zur Gestaltung und Formatierung. Diese Komponenten sind im Paket **Webserver** organisiert und befinden sich in den Verzeichnissen „**templates**“ und „**static**“.

5.5.1 Dynamische Datendarstellung mit JavaScript

Die Funktionalität des Frontends wird durch ein JavaScript-Skript „**script.js**“ gesteuert.

Die Benutzerdaten werden auf die Seite dynamisch dargestellt. Beim Laden der Seite werden die Benutzerdaten auf dem Server abgefragt und lokal gespeichert. Anschließend werden die Elemente der Oberfläche basierend auf diese Daten dynamisch erzeugt.

Asynchronen Serveranfragen

Da sowohl das Backend als auch die Datenbank asynchron implementiert sind, müssen auch die Anfragen im Frontend asynchron verarbeitet werden. Dies wird durch die Verwendung von asynchronen HTTP-Anfragen mittels der **fetch**-API realisiert. Dadurch wird verhindert, dass die Benutzeroberfläche während der Kommunikation mit dem Server blockiert wird. Antworten des Servers werden mittels Promises verarbeitet, wodurch das Frontend flexibel auf unterschiedliche Antwortzeiten reagieren kann.

Im Listing 9 ist die Funktion **get_user_data** dargestellt. Diese Funktion dient als Beispiel für die asynchrone Verarbeitung von Server-Anfragen im Frontend. Sie nutzt **async** und **await**, um eine GET-Anfrage an die Route „**/get_user_data**“ zu senden. Dabei wird das Token aus dem „**localStorage**“ als Bearer-Token im Authorisation-Header der Anfrage übermittelt.

Die Serverantwort wird in einem **try-catch**-Block verarbeitet, um potenzielle Fehler abzufangen und die Anwendung vor unerwarteten Abstürzen zu schützen. Falls die Antwort des Servers nicht den Status **ok** aufweist, wird eine Fehlermeldung generiert. Anschließend wird der zurückgegebene JSON-Inhalt geprüft. Falls ein Fehler im Antwortobjekt enthalten ist, wird dieser ebenfalls als Exception geworfen. Nur wenn die Serverantwort gültig ist und keine Fehlermeldung enthält, wird das empfangene Benutzerobjekt zurückgegeben. Im Fehlerfall wird die Funktion **errorHandler()** aufgerufen, um eine entsprechende Fehlerbehandlung vorzunehmen.

```
async function get_user_data() {
  try {
    const response = await fetch(
      '/get_user_data',
      {
        method: 'GET',
        headers: {
          'auth': `Bearer ${localStorage.getItem('token')}`
        }
      }
    );
    if (!response.ok)
      throw new Error(response.stat);
    else {
      const response_data = await response.json();
      if (response_data.error)
        throw new Error(response_data.error);
      else
        return response_data;
    }
  } catch (error) {
```

```

        errorHandler();
    }
}

```

Listing 9: script.js: get_user_data

Benutzerinteraktion Die Reaktionen auf die Benutzerinteraktionen sind mittels Event-Listener implementiert. Event-Listener ermöglichen es, auf Benutzerinteraktionen wie Klicks, Tastendrücke oder Mausbewegungen zu reagieren. In JavaScript werden sie mit der Funktion `addEventListener` an ein HTML-Element gebunden. Dabei wird das gewünschte Ereignis und eine Callback-Funktion angegeben, die beim Auftreten des Ereignisses ausgeführt wird.

Event-Listener können nicht nur für einzelne Elemente, sondern auch für übergeordnete Container genutzt werden. Dies ist besonders nützlich, wenn neue Elemente dynamisch hinzugefügt oder gelöscht werden.

Das Listing 10 zeigt den Prozess der Erstellung eines Formulars zum Hinzufügen eines neuen Hauses. In der ersten Zeile wird ein Event-Listener auf das Element mit der ID „BTNcreateHouse“ gesetzt. Beim Anklicken wird die Funktion `mnCreateHouse` aufgerufen, die das Eingabeformular erstellt. Dieser Funktion wird als Parameter ein `div`-Element übergeben, in dem das Formular angezeigt wird. Die HTML-Struktur des Formulars wird in den Zeilen 8 bis 15 definiert.

Anschließend wird ein Event-Listener für die Schaltfläche „BTNcloseHouseMN“ registriert, der das Formular schließt, indem er den Inhalt des `div`-Elements entfernt und dessen CSS-Klassen zurücksetzt.

Die Schaltfläche „BTNaddHouse“ erhält einen asynchronen Event-Listener, der die Funktion `addHouse` aufruft. Als Parameter wird der Name des Hauses übergeben, den der Benutzer im Eingabefeld „houseName“ eingegeben hat.

```

1      document.getElementById("BTNcreateHouse").addEventListener("click",
2          () => {
3              mnCreateHouse(document.getElementById("MNcreateHouse"));
4          });
5
6      function mnCreateHouse(parent_div){
7          parent_div.innerHTML = `
8              <div class="p-4">
9                  <label for="houseName">House Name:</label>
10                 <input type="text" id="houseName" value="House 1">
11                 <div>
12                     <button id="BTNaddHouse">Save</button>
13                     <button id="BTNcloseHouseMN">Close</button>
14                 </div>
15             </div>
16         `;
17
18         BTNcloseHouseMN.addEventListener("click", () => {
19             parent_div.innerHTML = "";
20             parent_div.classList = "";
21         });
22
23         BTNaddHouse.addEventListener("click", async () => {
24             await addHouse(document.getElementById("houseName").value);
25         });
26     }

```

Listing 10: script.js: mnCreateHouse

Die Implementierung dieser Event-Listener folgt einem einheitlichen Muster, das für weitere Schaltflächen und Formulare in der Anwendung übernommen werden kann. Je nach Anforderung kann die Funktionalität entsprechend angepasst und erweitert werden.

Oberfläche

Die Datei „`index.html`“, die sich im Verzeichnis `templates` befindet (Listing 11), bildet die Grundlage für die Benutzeroberfläche der Anwendung. Sie wird vom Server als Antwort auf einer GET-Anfrage an die Root-Route „/“ bereitgestellt.

Im `head`-Bereich der Datei sind der Seitentitel sowie die Verknüpfung zur CSS-Datei „`output.css`“ definiert, die das Styling der Anwendung steuert.

Der `body`-Bereich der HTML-Datei enthält zwei zentrale `div`-Elemente: „`navbar`“ für die Navigationsleiste und „`app`“ für den dynamisch generierten Inhalt der Seite. Am Ende der Datei ist ein Verweis auf das JavaScript-Skript „`script.js`“ eingebunden. Dieses Skript übernimmt die Steuerung des Routings sowie die Interaktion mit dem Benutzer, indem es die Ansicht basierend auf dem Authentifizierungsstatus des Nutzers und geladenen Daten dynamisch aktualisiert.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Baida Bachelor</title>
  <link rel="stylesheet" href="{{url_for('static',path='/output.css')}}">
</head>
<body>
  <div id="navbar"></div>
  <div id="app" class="app content"></div>
  <script src="../../static/js/script.js"></script>
</body>
</html>
```

Listing 11: `index.html`

Anmeldung und Registrierung

Beim Laden der Seite überprüft die Anwendung, ob ein gültiges Token im `localStorage` vorhanden ist. Falls kein Token existiert oder es ungültig ist, wird der Nutzer automatisch zur Login-Seite weitergeleitet. Andernfalls werden die Benutzerdaten von dem Server abgefragt und die Hauptseite wird geladen.

Die Login-Seite wird in der Funktion „`renderLoginForm()`“ (Listing 12) erstellt. Dabei wird das `div`-Element „`app`“ mit einem Formular versehen, das Eingabefelder für den Benutzernamen und das Passwort enthält (Zeile 1-18).

Die Schaltfläche „Login“ ist mit einem Event-Listener für das Ereignis „`submit`“ verknüpft. Durch Drücken des Login-Buttons werden die Daten aus den Eingabefeldern abgelesen und in den Variablen „`username`“ und „`password`“ gespeichert (Zeile 22-23). Anschließend wird eine asynchrone Post-Anfrage an die Route „`/login`“. Im `header`-Objekt wird der `Content-Type` auf „`application/json`“ gesetzt, damit der Server weiß, dass die übermittelten Daten im JSON-Format vorliegen (Zeile 30). Die Anmeldedaten werden mit `JSON.stringify()` in eine JSON-Zeichenkette umgewandelt und im `body` der Anfrage übermittelt (Zeile 32).

Nach dem Senden der Anfrage wartet die Anwendung mit `await` auf die Antwort des Servers. Sobald die Antwort eingetroffen ist, wird sie mit der Funktion `response.json()` in ein JSON-Objekt umgewandelt und in der Variablen `response_data` gespeichert (Zeile 35).

Stimmen die übermittelten Anmeldedaten mit den hinterlegten Benutzerinformationen überein, antwortet der Server mit einem JWT-Token. Dieses Token wird im localStorage gespeichert, wodurch der Benutzer authentifiziert und angemeldet wird (Zeile 37). Andernfalls antwortet der Server mit einem Fehler, dass die falschen Daten eingegeben wurden und die entsprechende Meldung wird ausgegeben.

```
1  app.innerHTML = `
2  <div class="flex items-center justify-center h-screen">
3    <h2>LOGIN</h2>
4    <form id="loginForm" class="space-y-4">
5      <div>
6        <label for="username">Username</label>
7        <input type="text" id="username">
8      </div>
9      <div>
10       <label for="password">Password</label>
11       <input type="password" id="password">
12     </div>
13     <button type="submit">LOGIN</button>
14   </form>
15   <div>
16     <button id="BTNsignup">SignUp here</button>
17   </div>
18 </div>`;
19
20 document.getElementById('loginForm').addEventListener('submit',
21   async (event) => {
22     event.preventDefault();
23     const username = document.getElementById('username').value;
24     const password = document.getElementById('password').value;
25
26     // asynchrone Anfrage auf die Route "login"
27     try {
28       const response = await fetch('/login', {
29         method: 'POST',
30         headers: {
31           'Content-Type': 'application/json'
32         },
33         body: JSON.stringify({ username, password })
34       });
35       //
36       const response_data = await response.json();
37       if ('token' in response_data) {
38         localStorage.setItem('token', response_data['token']);
39         appRouter();
40       }
41     } catch(error){ errorHandler(error);}
```

Listing 12: script.js: renderLoginForm

Unter dem Login-Button befindet sich ein Link zur Registrierungsseite. Ein Klick auf diesen Link ruft die Funktion `renderSignUpPage` auf, die das `div`-Element „app“ mit einem Formular für die Benutzerregistrierung füllt. Das Formular enthält Eingabefelder für den Benutzernamen, die E-Mail-Adresse und das Passwort. Durch Drücken der Taste „Sign Up“ werden die Anmeldedaten an die Route „/sign_up“ mittels einer Post-Anfrage gesendet. Wenn die Registrierungsdaten gültig sind, antwortet der Server mit einem JWT-Token und der Benutzer wird im System registriert.

Navigationsleiste

Die Funktion `renderNavbar` füllt das `div`-Element „navbar“ mit einer Navigationsleiste. Diese enthält einen Link zur Hauptseite sowie die Schaltflächen „My Devices“, „My Scenarios“ und „Logout“.

Ein Klick auf „Logout“ löscht das gespeicherte Token und leitet den Benutzer zur Anmeldeseite weiter. Die Schaltfläche „My Devices“ ruft die Funktion `renderMyDevicePage` auf, die die gespeicherten Geräte des Benutzers anzeigt. Mit „My Scenarios“ wird die Funktion `renderMyScenarioPage` aufgerufen, die die vom Benutzer erstellten Szenarien darstellt.

Hauptseite

Nach erfolgreicher Anmeldung und Erhalt des JWT-Tokens wird der Benutzer zur Hauptseite der Anwendung weitergeleitet. Dort erfolgt eine Anfrage an die Route „/get_user_data“, um die relevanten Benutzerdaten vom Server abzurufen. Die Serverantwort auf diese Anfrage besteht aus einem JSON-Objekt, das Listen der gespeicherten Häuser, Geräte und Szenarien enthält. Diese Daten werden anschließend in der lokalen Variablen „stored_data“ gespeichert und zur dynamischen Darstellung der Benutzeroberfläche verwendet. Falls die Token-Validierung fehlschlägt, wird der Benutzer automatisch zur Anmeldeseite zurückgeleitet. Nach Erhalt der Benutzerdaten wird die Hauptseite generiert.

Die Funktion `renderMainPage` füllt das `div`-Element „app“ mit der im Listing 13 dargestellten HTML-Struktur.

```
<div id="main_page">
  <div id="main_header">
    <p>Hello, ${data.user_data.username}!</p>
    <button id="BTNcreateHouse" type="button">Create New House</button>
  </div>
  <div id="MNcreateHouse"></div>
  <div id="houseList"></div>
</div>
```

Listing 13: html: renderMainPage

Oben auf der Seite wird eine Willkommensnachricht zusammen mit einer Schaltfläche zum Erstellen eines neuen Hauses angezeigt. Durch Anklicken des Buttons wird das `div`-Element „MNcreateHouse“ mit einem Formular befüllt, in das der Benutzer den Namen des Hauses eingeben kann. Mit der Schaltfläche „Save“ wird das Haus gespeichert. Dabei wird eine asynchrone Anfrage an die Route „add_house“ gesendet, die das Token und den Hausnamen enthält, und das Haus wird in der Datenbank gespeichert. Nach Abschluss wird die Seite neu geladen und das neue Haus erscheint.

In das `div`-Element „houseList“ werden die vom Benutzer erstellten Häuser, einschließlich der Räume und zugehörigen Geräte, geladen. Diese Daten sind in der lokalen Variablen „stored_data“ unter dem Schlüssel „houses“ gespeichert. Für jedes Haus wird ein `div`-Element mit der im Listing 14 dargestellten HTML-Struktur erstellt und an das `div`-Element „houseList“ angehängt. Dabei wird jede ID um die ID des jeweiligen Hauses erweitert, um die spätere Zuordnung der Räume und Geräte zu ermöglichen.

```
<div id="header${house.id}">${house.name} </div>
<div id="house_element${house.id}">
  <div>
    <button id="BTNcreateRoom${house.id}">New Room</button>
    <div id="create_room${house.id}"></div>
    <div id="room_list${house.id}"></div>
  </div>
  <button id="BTNdeleteHouse${house.id}">Delete House</button>
```

```
</div>
</div>
```

Listing 14: html: house_element

Im **header**-Element wird der Name des Hauses angezeigt. Unterhalb des Headers befindet sich eine Schaltfläche zum Anlegen eines neuen Raumes sowie der Bereich für die Auflistung der Räume. Zusätzlich gibt es eine Schaltfläche zum Löschen des Hauses. Der Button „create_room“ öffnet ein Formular, in dem der Benutzer den Namen des Raumes eingeben und speichern kann.

Anschließend wird das **div**-Element „room_list“ mit den Räumen des Hauses gefüllt. Dieser Vorgang ist ähnlich wie bei der Anzeige der Häuser. Für jeden Raum wird ein **div**-Element „room_element“ mit der ID des Raumes erzeugt. Dieses Element enthält den Namen des Raumes, eine Schaltfläche zum Hinzufügen eines Gerätes, den Bereich zur Auflistung der Geräte und eine Schaltfläche zum Löschen des Raumes. Die Schaltfläche zum Hinzufügen eines Geräts öffnet ein Formular, in das der Benutzer die ID und den Namen des Gerätes eingeben soll. Durch Bestätigen der Schaltfläche „Speichern“ wird das Gerät gespeichert und dem Raum zugeordnet.

Die zu dem Raum zugeordneten Geräte müssen auch angezeigt werden. Für jedes Gerät wird ein **div**-Element „device_element“ mit der ID des Gerätes erzeugt. Im Listing 15 ist die HTML-Struktur des Elements angezeigt. Dieses Element enthält der Name und ID des Geräts, eine Schaltfläche zum Löschen des Gerätes und zwei weitere **div**-Elemente für die Darstellung der von Gerät gesendeten Daten und den gespeicherten Befehlen, die das Gerät ausführen kann.

```
<div id="device_element${device.dev_id}"
  <div>
    <p>NAME: ${device.name}</p>
    <p> ID: ${device.dev_id} </p>
    <div id="deviceData${device.dev_id}"></div>
  </div>
  <div id="commands${device.dev_id}"></div>
  <button id="deleteDevice${device.dev_id}"> DELETE </button>
</div>
```

Listing 15: html: device_element

WebSocket

Im **div**-Element „deviceData“ werden die vom Gerät empfangenen Daten angezeigt. Dazu wird eine WebSocket-Verbindung aufgebaut. Das Listing 16 zeigt die Initialisierung der Verbindung sowie die Verarbeitung der eingehenden Daten.

Zunächst wird ein WebSocket-Objekt mit der URL der WebSocket-Route erzeugt (Zeile 1-3). Sobald die Verbindung erfolgreich hergestellt wurde, sendet das Skript das Authentifizierungstoken an den Server (Zeile 5-8), um die Verbindung zu autorisieren.

Bei jeder eingehenden Nachricht wird das Ereignis **onmessage** ausgelöst (Zeile 10). Die empfangenen Daten werden in ein JavaScript-Objekt umgewandelt (Zeile 11) und anschließend wird das **div**-Element mit der ID „deviceData“ für das entsprechende Gerät abgerufen (Zeile 12).

Wenn das Gerät keine vordefinierten Datenfelder hat, werden die Schlüssel des empfangenen Datenobjekts extrahiert und als Datenfelder verwendet (Zeilen 13-16). In diesem Fall werden alle empfangenen Daten angezeigt.

Für jedes dieser Datenfelder wird geprüft, ob es im empfangenen Datenobjekt vorhanden ist (Zeile 17-18). Ist dies der Fall, sucht das Skript nach einem **p**-Element, dessen ID den Namen des Datenfelds und die Geräte-ID enthält (Zeile 19-20). Ist ein solches Element nicht vorhanden,

wird es dynamisch erzeugt und in das `div`-Element eingefügt (Zeile 21-25). Schließlich wird der Inhalt des Elements mit dem Schlüssel und dem entsprechenden Wert aus den empfangenen Daten aktualisiert (Zeile 26).

Wenn die Verbindung unterbrochen wird, wird eine Meldung an die Konsole ausgegeben.

```
1  const ws = new WebSocket(  
2    `ws://127.0.0.1:8000/mqtt/device/${device.dev_id}`  
3  );  
4  
5  ws.onopen = () => {  
6    ws.send(JSON.stringify({ auth: "Bearer " +  
7      localStorage.getItem("token") }));  
8  };  
9  
10 ws.onmessage = (event) => {  
11   const data = JSON.parse(event.data);  
12   const parent_div = device_element.querySelector(`#deviceData${  
13     device.dev_id}`);  
14   let data_fields = device.data_fields;  
15   if (data_fields.length == 0) {  
16     data_fields = Object.keys(data);  
17   }  
18   for (const key of data_fields) {  
19     if (data.hasOwnProperty(key)) {  
20       let dataField = device_element.querySelector(  
21         `#${key}${device.dev_id}`);  
22       if (!dataField) {  
23         dataField = document.createElement('p');  
24         dataField.id = `${key}${device.dev_id}`;  
25         parent_div.appendChild(dataField);  
26       }  
27       dataField.innerHTML = `${key}: ${data[key]}`  
28     }  
29   };  
30  
31 ws.onclose = () => {  
32   console.error(`WebSocket for device ${device.dev_id} closed`);  
33 };
```

Listing 16: script.js: websocket

5.5.2 Styling

6 Ergebnisse und Diskussion

Hier wird zusammengefasst, was ich mit diesem Projekt gelernt habe, welche Ziele erreicht und nicht erreicht habe, mögliche Weiterentwicklung des Systems etc

7 Quellen

Literatur

- [1] O. Baida, *Anbindung der Sensoren und Aktoren an den Arduino zur Realisierung eines Sicherheitssystems*, Projektarbeit 1, 2024.
- [2] O. Baida, Projektarbeit 2 *Sicherheitssystem für das Haus basierend auf Arduino, ESP8266 & Raspberry Pi* <https://github.com/oleksiibaida/PA2.git>
- [3]
- [4] Statista, „*Smart Home - Anzahl der Haushalte in Deutschland 2028*“, Zugriffen: 13. Januar 2025. [Online]. Verfügbar unter <https://de.statista.com/prognosen/885611/anzahl-der-smart-home-haushalte-in-deutschland>
- [5] J. Breithut, „*Strom und Heizung: Wann ein Smart Home wirklich beim Energiesparen hilft*“, Der Spiegel, 17. Juli 2022. Zugriffen: 13. Januar 2025. [Online]. Verfügbar unter: <https://www.spiegel.de/netzwelt/gadgets/strom-und-heizung-wann-ein-smart-home-wirklich-beim-energiesparen-hilft-a-ffb4b710->

Links zur verwendeten Hardware:

- [6] Arduino.cc, *Arduino UNO*, <https://docs.arduino.cc/hardware/uno-rev3/>
- [7] Raspberry Pi Foundation, *Raspberry Pi 1 B+*, <https://www.raspberrypi.com/products/raspberry-pi-1-model-b-plus/>
- [8] Espressif, *ESP8266*, <https://www.espressif.com/>, <https://www.electronicwings.com/sensors-modules/esp8266-wifi-module>
- [9] Bosh, *BME680 - Datasheet*, <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme680-ds001.pdf>, Zugriff: 4. Februar 2025.
- [10] Vishay, *VCNL4040 - Datasheet*, <https://www.vishay.com/docs/84274/vcnl4040.pdf>, Zugriff: 5. Februar 2025.

Links zur verwendeten Software:

- [11] Dr Andy Stanford-Clark, Arlen Nipper, *Message Queuing Telemetry Transport*, <https://mqtt.org/>
- [12] Guido van Rossum, Python Software Foundation, *Python*, <https://www.python.org/>
- [13] Telegram FZ-LLC, *Telegram Messenger*, <https://github.com//telegramdesktop/tdesktop>

Linux-Paketete:

- [14] Jouni Malinen, *hostapd*, <https://w1.fi/hostapd/>, Zugriff am: 19. September 2024.
- [15] Simon Kelley, *dnsmasq*, <https://dnsmasq.org/doc.html>, Zugriff am: 20. September 2024.
- [16] Eclipse Foundation, *Eclipse Mosquitto*, <https://mosquitto.org/>

ESP- und Arduino-Bibliotheken

- [17] Knolleary, *PubSubClient*, <https://pubsubclient.knolleary.net/>, Zugriff am: 21. Oktober 2024.
- [18] ESPWIFI.h, <https://arduino-esp8266.readthedocs.io/en/latest/esp8266wifi/readme.html>
- [19] EEPROM.h, <https://docs.arduino.cc/learn/built-in-libraries/eeprom/>
- [20] Keypad.h <https://docs.arduino.cc/libraries/keypad/>
- [21] R. Scholz, *Syncloop*, Persönliche Mitteilungen
Python-Bibliotheken
- [22] Pierre Fersing, Roger Light *paho-mqtt*, <https://pypi.org/project/paho-mqtt/>, Zugriff am: 21. Oktober 2024.
- [23] Open Source, *python-telegram-bot*, <https://docs.python-telegram-bot.org/en/v21.6/>
- [24] Python Software Foundation, *json*, <https://docs.python.org/3/library/json.html>
- [25] Python Software Foundation, *threading*, <https://docs.python.org/3/library/threading.html>
- [26] Python Software Foundation, *queue*, <https://docs.python.org/3/library/queue.html>
- [27] Gerhard Häring, *sqlite3*, <https://docs.python.org/3/library/sqlite3.html>
- [28] Lawrence Hudson, *pyzbar*, <https://github.com/NaturalHistoryMuseum/pyzbar/>
- [29] Intel, *OpenCV*, <https://github.com/opencv/opencv-python>
- [30] Aio-Libs, *aiohttp*, <https://github.com/aio-libs/aiohttp>

Abbildungsverzeichnis

1	Datenbank	19
---	---------------------	----

Tabellenverzeichnis

1	MQTT-Nachrichten	7
2	Technische Daten des Arduino Uno	8
3	Technische Daten des ESP8266	9
4	Technische Daten des ESP32	9
5	Technische Daten des BME680	10
6	Technische Daten des VCNL4040	11

Programmcode

1	ESP: connect_wifi_mqtt()	13
2	ESP: Access Point	14
3	ESP: setup_ap	14
4	ESP: mqtt_connect	15
5	ESP: mqtt_connect	16
6	M5Stick: Sensoren	17
7	M5Stick: vcnl_setup	17
8	db: base.py	21
9	script.js: get_user_data	29
10	script.js: mnCreateHouse	30
11	index.html	31
12	script.js: renderLoginForm	32
13	html: renderMainPage	33
14	html: house_element	33
15	html: device_element	34
16	script.js: websocket	35